



WINDMILL

Order Book Auction-Based Exchange

&

Karma

A Self-Sovereign Prediction-Market Oracle



Google
Summer of Code

CANDIDATE DETAILS

Candidate	Aniket Kumar
Email	iamaniketkumarmaner@gmail.com
GitHub	https://github.com/aniket866/
Discord	aniket866
University	Panjab University, Chandigarh — B.E. Information Technology (3rd Year, CGPA 8.3)
Project Size	Large (350 Hours) — 22 Weeks

02 PROJECT BACKGROUND & MY RESEARCH CONTEXT

Before going deep into the project details let's see what we have already done and what's the role of those works/projects in this project.

Project	Type	Pricing Model	Limitation Being Solved
Hammer	Auction Platform	Time-decay	No order book, only auctions
Maelstrom	AMM Pool	Auction-based	Needs LP tokens, no direct orders
Windmill	Order Book DEX	Per-order	Combines orders + auctions

PROBLEM STATEMENT:

Let's see what is lacking in the current crypto trading system:

Currently trading crypto is like buying from a shop where either the shop decides the price (AMM) or we have to keep changing our own price again and again and currently both seems to be inefficient.

In more simple terms:

There are only two ways right now. In one way the system itself tells us the price. we don't get to choose much and we just accept it. If we try to buy more then the price suddenly becomes worse and we will be paying more than we expected to be paid.

PROBLEM Analogy:

Imagine a small market with two people:

- **buyer:** I will buy this phone for 100 rupees"
- **Seller :** I wil sell it for 120 rupees.

Now what happens? **system decides price**

buyer goes to a shop where the shopkeeper says:

Price is 115 rupees. Take it or leave it.

- buyer wants cheaper : but has no control

If he buys more price increases

He feels forced and may goes to overpay.

How is this inefficient?

Currently this Bargaining

Is done by
Manual pricing

Buyer says 100, seller says 120
No deal happens

They keep changing prices again
and again:
105 rupees → 110 rupess → 112
rupees and soon ...

This takes time, effort, and cost.

PROJECT OBJECTIVE:

WHAT WE ARE BUILDING

We are building a new crypto exchange system, which runs totally on Blockchain.

Hence there is no company, no server, no middleman at all.

The main idea is very simple: when we place an order the price does not stay fixed. It changes over time. If we want to buy our price slowly decreases. If we want to sell, the price slowly increases.

After some time, the buyer's price and the seller's price become the same. At that moment, the trade happens automatically on the blockchain without anyone needing to manually interfere.

THE THREE GOALS

- * Give users full price control
- * No off-chain server or middleman
- * Gas efficient enough to actually use

WHAT IS SOLVED

Right now every DEX either uses a pool or a fixed price order book. both have problems.

AAM system picks the price. we have no say. buy too much and the price jumps on us.

Order book: we pick a price but it never moves. prices keep changing in the market. our order sits there doing nothing.

Windmill is different because the price moves with time. it finds its own match. we place it once and let it run.

Trade happens automatically when prices match

Less effort, more control, and better efficiency

THE CORE INNOVATION

- No need to accept system-decided prices
- No need to manually change prices again and again
- Prices adjust automatically over time

HOW IS SOLVED

Each order is stored as a moving price using a start price and a slope, so the price changes automatically with time using on-chain math.

When a buyer's price becomes equal to or higher than a seller's price, the contract detects it and matches the orders at a fair price.

All logic runs fully on-chain, and matching can be triggered by anyone, but the contract always verifies everything itself.

Matching is triggered either by a user or a helper bot

No off-chain server is trusted, and no background process is required.

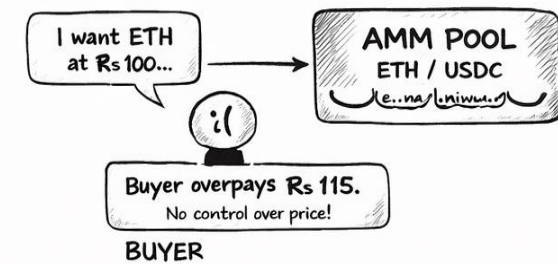
THE TECHNICAL APPROACH

- Only two orders are checked at a time.
- Remaining order continues with same price after partial fill .
- System works even without bots, but bots can help trigger matches faster

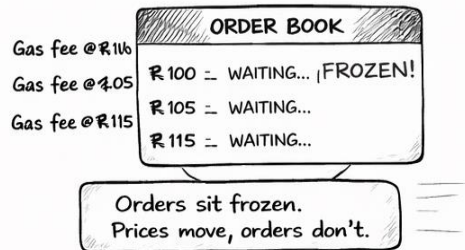
"Two price curves moving toward each other and when they cross, a trade happens. no middleman. just math."

PROBLEM vs SOLUTION

TODAY – THE PROBLEM

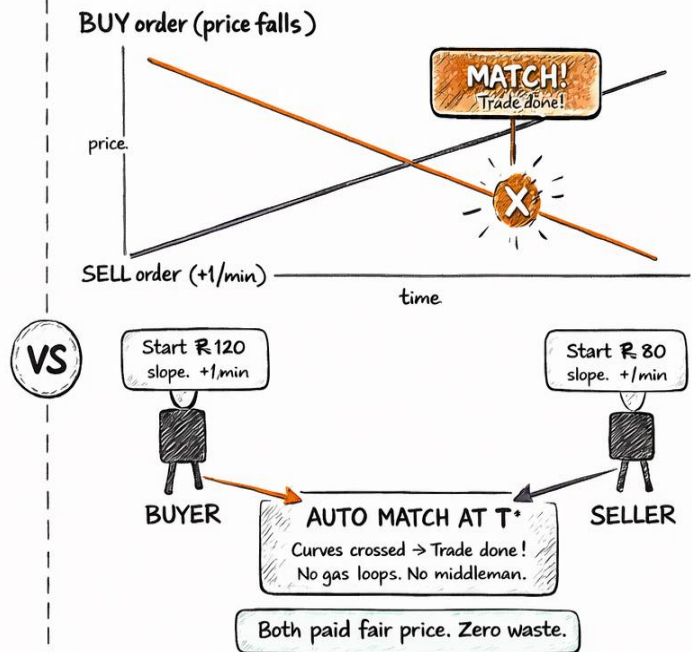


ORDER BOOK – ALSO BROKEN



RESULT: No fair trade. Gas wasted.
User loses.

WINDMILL – THE FIX



Why AMMs Are Not Enough

AMM Limitations (Maelstrom model)

- Price guarantee is impossible hence slippage always exists
- Users must deposit into pool and hold LP tokens
- Impermanent loss is a real economic risk for LPs
- Individual order control does not exist
- Invariant function determines price, not trader intent

Order Book Advantages

- Exact fill prices are known before submission
- No LP tokens so users always hold their own assets
- Limit orders give full price control to traders
- Compatible with auction dynamics (Windmill's key innovation)
- More expressive thus different strategies per order

02 SYSTEM ARCHITECTURE

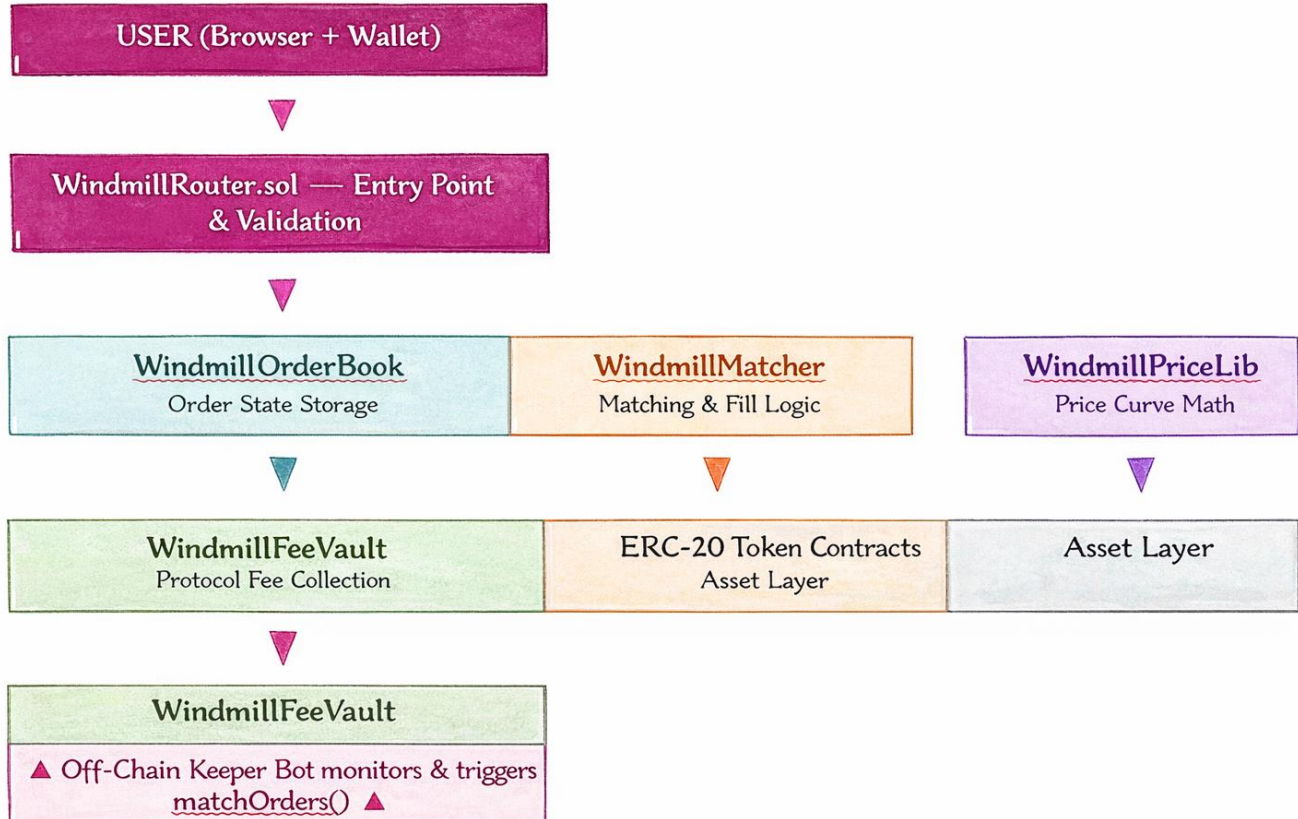
Windmill is a tri-repository system: Solidity smart contracts (on-chain settlement), an EVM Keeper Bot (off-chain automation), and a Next.js WebUI (user interface). Together they form a fully trustless, order-book auction exchange on EVM-compatible chains..

WINDMILL - SOLIDITY	WINDMILL - EVM - KEEPER	WINDMILL - WEBUI
Languages: Solidity 0.8.x Framework: Hardhat + Foundry Contracts: 6 modular contracts Tests: Mocha/Chai + Foundry Forge Security: Slither + Mythril	Languages: Node.js 20+ / JS ESM Library: Ethers.js v6 Pattern: Strategy-based keeper Config: .env + DRY_RUN mode CI: GitHub Actions included	Languages: Next.js 14 + TypeScript Wallet: RainbowKit + wagmi Styling: Tailwind CSS Indexing: The Graph (subgraph) Deploy: IPFS / Vercel

Key Principle: The smart contracts are the single source of truth. The Keeper and WebUI are stateless off-chain helpers and they read events and call contract functions but hold no privileged data.

System Architecture Layers:

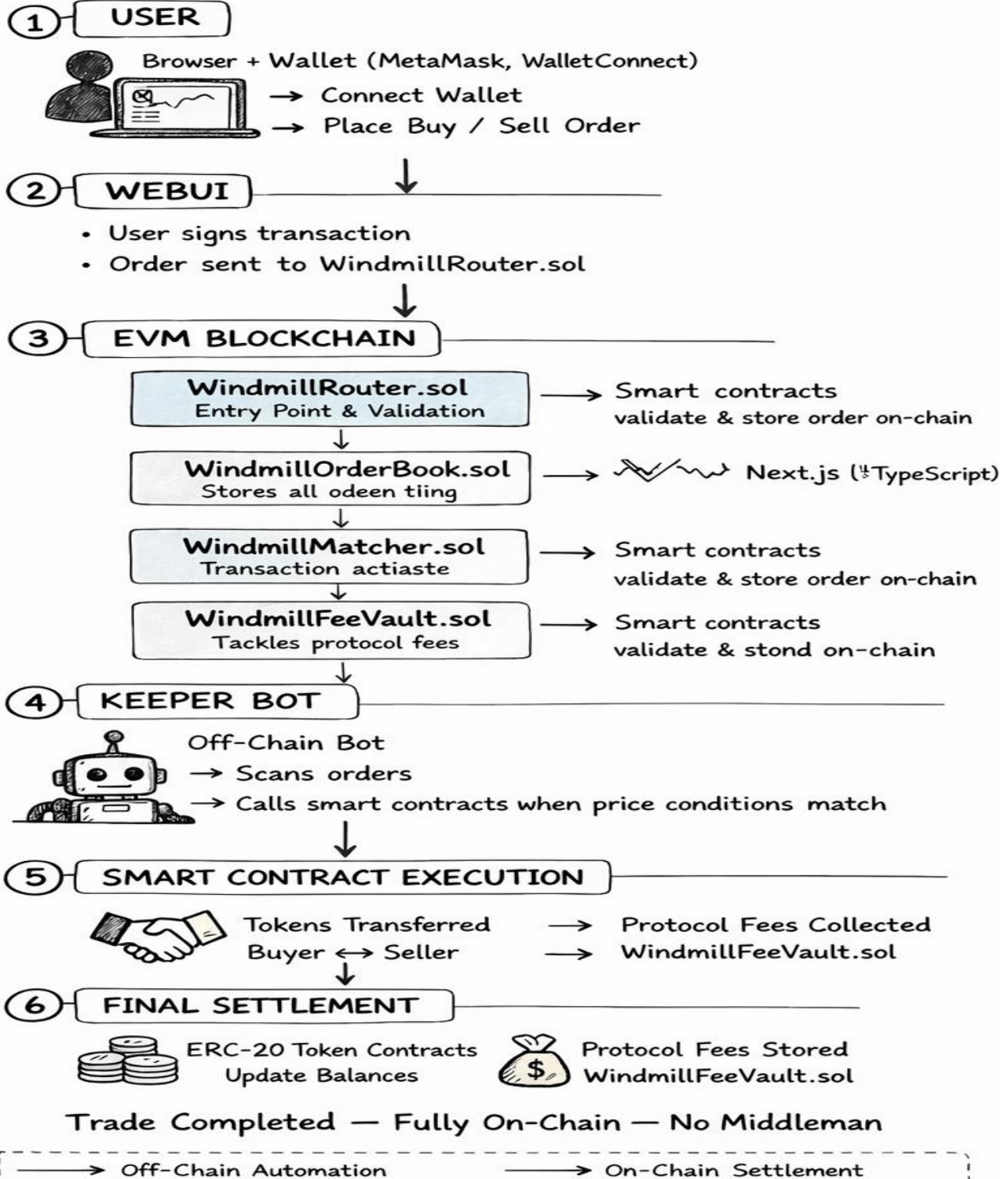
The full system spans three layers: User → Smart Contract Layer → Asset/Fee Layer. The off-chain Keeper Bot sits alongside, polling for matchable order pairs and submitting settlement transactions.



WINDMILL – SYSTEM ARCHITECTURE

Solidity Smart Contracts + EVM Keeper Bot + Next.js WebUI + Next.js W

Single Source of Truth → Smart Contracts (on-chain)



Layer Summary:

Layer	Component	Role
User Layer	Browser wallet (MetaMask / WalletConnect)	Submits signed transactions, reads WebUI state
Entry Layer	WindmillRouter.sol	Validates all inputs, locks ERC-20 tokens, routes to book
Logic Layer	OrderBook + Matcher + PriceLib	State, matching algorithm, price math
Settlement Layer	FeeVault + ERC-20	Token swap execution, fee deduction
Off-Chain Layer	Keeper Bot + The Graph	Monitoring, automation, event indexing

03 THE WINDMILL MECHANISM & CONTRACTS

Central point: In Windmill, every order is not a fixed price, it's more like a time changing curve. a buy order starts from high price and slowly decays with time. a sell order starts low and keeps rising.

when both curves meet or intersect, thats where trade happens. no off-chain engine, no oracle, no external calls involved. everything is fully deterministic and runs completely on-chain.

Contract	Responsibility	Key Functions
WindmillRouter.sol	User-facing entry point. Validates inputs, handles ERC-20 approvals, routes to OrderBook	placeOrder() cancelOrder() matchOrders() batchMatch()
WindmillOrderBook.sol	Core state store. All Order structs live here. Strict access control: only Matcher may modify	storeOrder() updateFill() deactivate() getOrder() getActiveOrders()
WindmillMatcher.sol	Matching logic. Checks price intersection, executes fills, transfers tokens	tryMatch() _executeFill() isMatchable() settleTransfer()
WindmillPriceLib.sol	Pure math library. No state, no storage. Price at any time T, intersection time T*, validity	currentPrice() intersectionTime() isExpired() isFillable()
WindmillFeeVault.sol	Fee collection. Takes a cut on each match. Governance-controlled withdrawal	depositFee() withdraw() setFeeRate() feeBalance()
WindmillFactory.sol	Deploys new trading pair instances Each pair has its own Router	createPair() getPair() allPairs()

Dependency Graph:

- WindmillFactory → deploys → WindmillRouter (one per trading pair)
- WindmillRouter → calls → WindmillOrderBook (store / cancel / query)
- WindmillRouter → calls → WindmillMatcher (trigger match)
- WindmillMatcher → calls → WindmillOrderBook (read + update fills)
- WindmillMatcher → calls → WindmillPriceLib (price check — pure, no SLOAD)
- WindmillMatcher → calls → WindmillFeeVault (deposit protocol fee)
- WindmillMatcher → calls → ERC-20 contracts (transferFrom / transfer)

Main functions and their roles:

It is the single public-facing entry point and all user interactions go through this contract. It does not store anything except some minimal routing state, all orders are stored in WindmillOrderBook.

placeOrder()

- Accepts order parameters from user wallet.
- Validates non-zero amount, positive startPrice, distinct token pair.
- Calls IERC20.transferFrom to lock user tokens.
- Delegates order storage to WindmillOrderBook.storeOrder().
- Emits OrderPlaced event.

cancelOrder()

- Verifies msg.sender is the order maker.
- Checks order is still active.
- Calls OrderBook.deactivate() and refunds unfilled tokenIn balance back to the maker.
- Emits OrderCancelled event.

matchOrders(buyId, sellId)

- Single-pair match call.
- Readable by any address including the Keeper Bot.
- Validates both orders are active and token pairs are complementary.
- Delegates price check and execution to WindmillMatcher.tryMatch().
- Can be called by anyone , no privilege required.

batchMatch(buyIds[], sellIds)

- Batches multiple order pairs into a single transaction.
- Loops over paired arrays and calls tryMatch() for each pair. Amortises base gas cost across N settlements.
- Used by the Keeper
- Bot to maximise efficiency per transaction.

Order Struct :

```
struct Order {
    uint256 id;           // unique order identifier
    address maker;        // wallet that created the order
    address tokenIn;      // ERC-20 token being offered
    address tokenOut;     // ERC-20 token wanted in return
    uint256 amountIn;     // original total amount offered
    uint256 amountFilled; // amount already matched so far
    uint256 startPrice;   // price per unit at createdAt (18 decimals)
    int256 priceSlope;    // price change per second (negative = buy, positive = sell)
    uint256 floorPrice;   // if buy: cancel when price falls below this
    uint256 ceilPrice;    // if sell: cancel when price rises above this
    uint256 createdAt;   // block.timestamp when order was placed
    bool isBuy;          // true = buy order, false = sell order
    bool isActive;       // false once filled or cancelled
}
```

It is a stateless pure library, with zero storage reads and zero state writes. All functions are pure computations. It is deployed once and then called by WindmillMatcher on every match attempt.

BUY ORDER : Decreasing Price Curve

$$P_{\text{buy}}(t) = \text{startPrice} - |\text{slope}| \times (t - \text{createdAt})$$

The buyer decreases the maximum they will pay over time.
Boundary: floorPrice — auto-cancel if price hits floor.

SELL ORDER : Increasing Price Curve

$$P_{\text{sell}}(t) = \text{startPrice} + \text{slope} \times (t - \text{createdAt})$$

The seller increases the minimum they will accept over time.
Boundary: ceilPrice — auto-cancel if price exceeds ceiling.

INTERSECTION means SETTLEMENT

When $P_{\text{buy}}(t) \geq P_{\text{sell}}(t)$: $\text{clearPrice} = (P_{\text{buy}} + P_{\text{sell}}) / 2$ | Tokens swap atomically at midpoint

Library Function Reference:

Function	Modifier	Description
currentPrice(order, t)	pure	Computes $P(t) = \text{startPrice} + \text{priceSlope} \times (t - \text{createdAt})$. Returns uint256 precision price.
intersectionTime(buy, sell)	pure	Solves $P_{\text{buy}}(t^*) = P_{\text{sell}}(t^*)$ algebraically. Returns t^* as block timestamp. If no future intersection, returns type(uint256).max .
isExpired(order, t)	pure	Returns true if $t > \text{order.expiry}$. Used by Router to auto-void stale orders without looping.
isFillable(order, t)	pure	Returns true if order is active AND not expired AND currentPrice is within floor/ceil bounds.
isMatchable(buy, sell, t)	pure	Combines currentPrice for both and returns true if $\text{buyPrice} \geq \text{sellPrice}$. Gas-efficient single call.

Why Pure Library Design Matters:

A pure library with no SLOAD calls means the price check itself costs only computation gas (200 gas) rather than storage access gas (2100 gas). thousands of match attempts per day and this is the single biggest gas optimisation in the system.

Fixed-Point Arithmetic

Aspect	Unit	Explanation
WAD precision	$1 \text{ WAD} = 1 \times 10^{18}$	All prices stored and computed in 18-decimal fixed-point. Prevents precision loss on sub-unit prices.
Slope units	WAD / second	priceSlope is expressed as WAD units per second.
Overflow guard	uint256 bounds	startPrice and slope are bounded by floor/ceil params to prevent overflow at extreme timestamps.
Midpoint formula	$(P_{\text{buy}} + P_{\text{sell}}) / 2$	Computed in uint256, no precision loss since both values are WAD-scale.

Price Curve Formula:

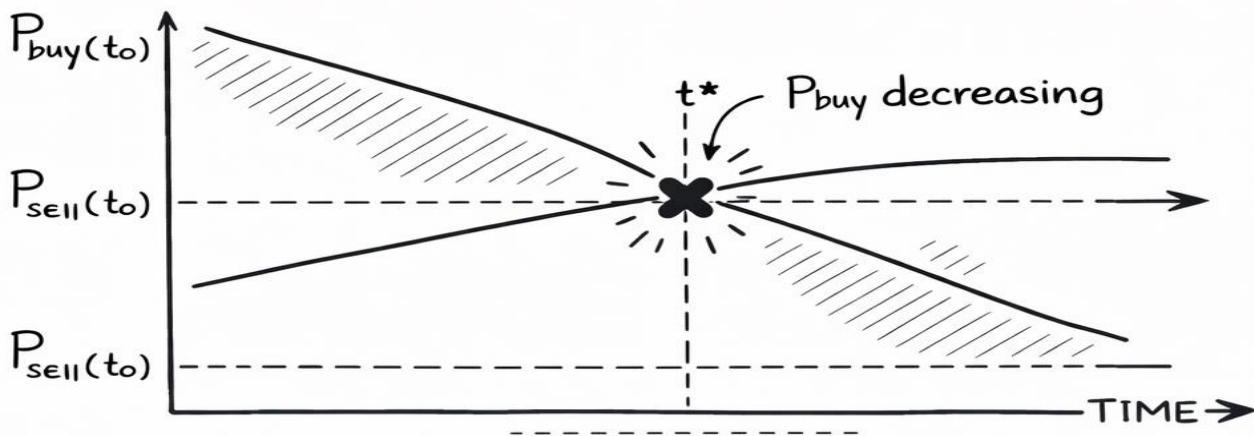
The core idea of Windmill: every order is not a static price but a time-dependent price. This section deals the mathematics, boundary conditions, and how intersection-based settlement works in real.

```
// WindmillPriceLib.sol
function currentPrice(
    Order memory o,
    uint256 timestamp
) public pure returns (uint256) {
    int256 elapsed = int256(timestamp - o.createdAt);

    int256 raw = int256(o.startPrice) + (o.priceSlope * elapsed);

    if (raw <= 0) return 0;
    uint256 p = uint256(raw);
    // enforce bounds
    if (o.isBuy && p < o.floorPrice) return o.floorPrice;
    if (!o.isBuy && p > o.ceilPrice) return o.ceilPrice;
    return p;
}
```

the price of any order at any moment in time is computed by a simple linear formula. this is pure on-chain arithmetic and no external calls needed:



INTERSECTION = SETTLEMENT

Settlement price = midpoint of $P_{buy}(t^*)$ and $P_{sell}(t^*)$

Intersection Calculation — Finding T*

the core mathematical: given a buy order B and sell order S the intersection time T* can be computed from their parameters:

```
// WindmillPriceLib.sol
// Returns intersection timestamp, or 0 if they never cross
function intersectionTime(
    Order memory buy,
    Order memory sell
) public pure returns (uint256) {

    int256 pBuy0 = int256(buy.startPrice) + buy.priceSlope *(int256(buy.createdAt));
    int256 pSell0 = int256(sell.startPrice) + sell.priceSlope * (-int256(sell.createdAt));

    int256 slopeDiff = buy.priceSlope - sell.priceSlope;
    if (slopeDiff == 0) return 0; // parallel lines, never intersect

    int256 tStar = (pSell0 - pBuy0) / slopeDiff;
    if (tStar <= int256(block.timestamp)) return 0; // already past

    return uint256(tStar);
}
```

Condition	Equation State	System Behaviour
Before t*	$P_{\text{buy}}(t) > P_{\text{sell}}(t)$	Not yet matchable, buyer still wants more than seller is asking (Dutch gaps)
At t*	$P_{\text{buy}}(t^*) = P_{\text{sell}}(t^*)$	Exact crossing and system settles immediately if Keeper or taker calls tryMatch()
After t*	$P_{\text{buy}}(t) < P_{\text{sell}}(t)$	Curves have crossed so buyer willing to pay less than seller is asking again (typical after midpoint)
At floor	$P_{\text{buy}}(t) = \text{floorPrice}$	Buy order auto-cancels thus buyer no longer willing to accept any price
At ceiling	$P_{\text{sell}}(t) = \text{ceilPrice}$	Sell order auto-cancels ,seller's demand exceeds market willingness

Why FIFO Becomes Irrelevant

as each order has a different priceSlope, two orders with the same price at time T1 will almost certainly have different prices at T2. the probability of a tie at matching time approaches zero. hence, traditional FIFO tiebreaking is mathematically unnecessary and takers simply choose which orders to match based on current prices.

The algorithmic heart of Windmill. Implements the intersection check, fill execution, partial fill logic, and token settlement.

Matching Algorithm — Step by Step

Step	Action	Detail
Step 1	Load both orders	Read buy and sell Order structs from WindmillOrderBook
Step 2	Validate pair	Confirm <code>buy.tokenIn == sell.tokenOut</code> and <code>buy.tokenOut == sell.tokenIn</code>
Step 3	Check both active	Both <code>isActive</code> flags must be true; both <code>isBuy</code> flags must differ
Step 4	Compute current prices	Call <code>PriceLib.currentPrice(order, block.timestamp)</code> for each, $O(1)$, pure
Step 5	Intersection check	Require <code>buyPrice >= sellPrice</code> — the matching condition
Step 6	Compute clearPrice	<code>clearPrice = (buyPrice + sellPrice) / 2</code> means midpoint fair to both parties
Step 7	Determine fill amount	<code>min(buy.remaining, sell.remaining)</code> so smaller order fully filled
Step 8	Deduct fee	<code>FeeVault.depositFee()</code> deducts protocol basis points from clearPrice
Step 9	Settle tokens	ERC-20 <code>transferFrom</code> router custody to respective makers
Step 10	Update state	<code>OrderBook.updateFill()</code> on both orders; <code>deactivate()</code> the fully-filled one

Partial Fill Behaviour:

When two orders of different sizes intersect, the smaller order is fully settled. The larger order has its `amountFilled` incremented and remains ACTIVE with the exact same `startPrice`, `priceSlope`, and `createdAt` and its price curve continues unchanged from where it left off.

Taker Route	Keeper Route	Batch Route
Any user may call <code>matchOrders()</code> directly. No keeper required. * Reward: traders see price difference and call <code>settle</code> to make profit.	The off-chain Keeper Bot scans active orders every cycle (configurable interval), identifies matchable pairs, and submits <code>batchMatch()</code> transactions automatically.	<code>batchMatch(buyIds[], sellIds[])</code> settles multiple pairs in one transaction. Reduces per-settlement gas cost significantly for high-volume markets.

Why Partial Fill Behaviour:

- Allows large orders to be executed gradually instead of waiting for a full match
- Keeps orders active in the market, improving chances of matching
- Reduces delays and avoids orders getting stuck at one price
- Improves overall liquidity and makes trading smoother and more efficient

WindmillMatcher.sol: (Expected code not final , yet to be discussed with mentor to finalize)

```
// WindmillMatcher.sol
function _executeFill(
    uint256 buyId,
    uint256 sellId,
    uint256 clearPrice
) internal {
    Order storage buy = orders[buyId];
    Order storage sell = orders[sellId];

    uint256 buyRemaining = buy.amountIn - buy.amountFilled;
    uint256 sellRemaining = sell.amountIn - sell.amountFilled;
    uint256 fillAmount = buyRemaining < sellRemaining
        ? buyRemaining
        : sellRemaining;

    buy.amountFilled += fillAmount;
    sell.amountFilled += fillAmount;

    // IMPORTANT: do NOT reset startPrice or priceSlope
    // the remaining portion continues on the same price trajectory
    if (buy.amountFilled == buy.amountIn) buy.isActive = false;
    if (sell.amountFilled == sell.amountIn) sell.isActive = false;

    emit OrderMatched(buyId, sellId, fillAmount, clearPrice, block.timestamp);
}
```

Some Events Related to order match and order place:

```
// Full event surface for indexers and frontend
event OrderPlaced(
    uint256 indexed orderId,
    address indexed maker,
    address tokenIn,
    address tokenOut,
    uint256 amountIn,
    uint256 startPrice,
    int256 priceSlope,
    uint256 timestamp
);

event OrderMatched(
    uint256 indexed buyId,
    uint256 indexed sellId,
    uint256 fillAmount,
    uint256 clearPrice,
    uint256 timestamp
);

event OrderCancelled(uint256 indexed orderId, address indexed maker, uint256 timestamp);
event OrderExpired(uint256 indexed orderId, uint256 finalPrice, uint256 timestamp);
event PartialFill(uint256 indexed orderId, uint256 filled, uint256 remaining);
```

FEE VAULT+FACTORY

WindmillFeeVault.sol collect protocol fees from every settled trade. WindmillFactory.sol let anyone create new trading pairs, any two ERC-20 tokens can make a market.

WindmillFeeVault.sol:

FEE COLLECTION ENGINE:

Function	Description
depositFee()	Called by Matcher after every settlement. Deducts basis points from clearPrice.
setFeeRate()	Governance-only. Sets fee in basis points (30 = 0.3%).
withdraw()	Owner-only. Withdraws accumulated fees to treasury address.
feeBalance()	Public view. Returns total protocol fee accumulated for a token.

WindmillFactory.sol

PAIR DEPLOYMENT ENGINE:

Function	Description
createPair()	Deploys a fresh set of Router + OrderBook + Matcher for a token pair. Returns pair address.
getPair()	Returns the Router address for a given (tokenA, tokenB) pair. Order-agnostic lookup.
allPairs()	Returns all deployed pair addresses. Used by WebUI pair discovery.

The Factory pattern means Windmill scales horizontally: each new ERC-20 pair gets its own isolated contract set with independent order books, price curves, and fee vaults. No pair can affect another.

Fee Rate Design

Parameter	Value	Rationale
Default Fee Rate	30 basis points (0.30%)	Competitive with Uniswap v2's 0.30% LP fee
Fee Receiver	Protocol Treasury (governance-controlled address)	Collected fees can be redistributed or burned
Fee Token	Same as settlement token (tokenOut of each trade)	No additional swap required to collect fees
Fee Deduction Point	During <code>_executeFill()</code> in Matcher, before token transfer	Atomic: fee and settlement happen in one transaction

ORDER LIFECYCLE AND STATE MACHINE

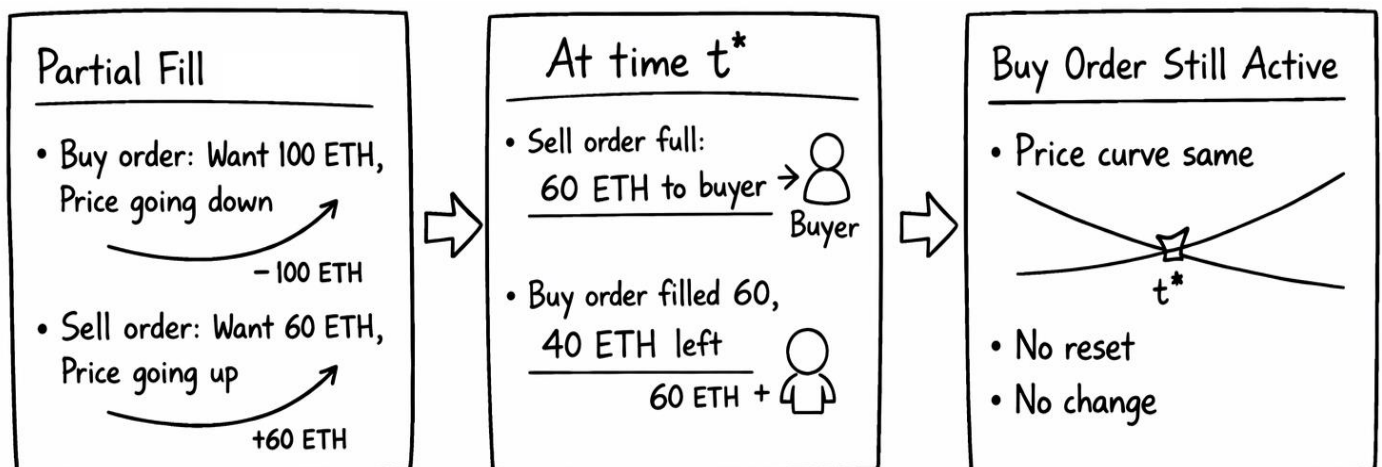
Every order in Windmill passes through a defined set of states. The state is tracked by the `isActive` flag and `amountFilled` field. Transitions are irreversible except via the cancel path.



State Transition Rules

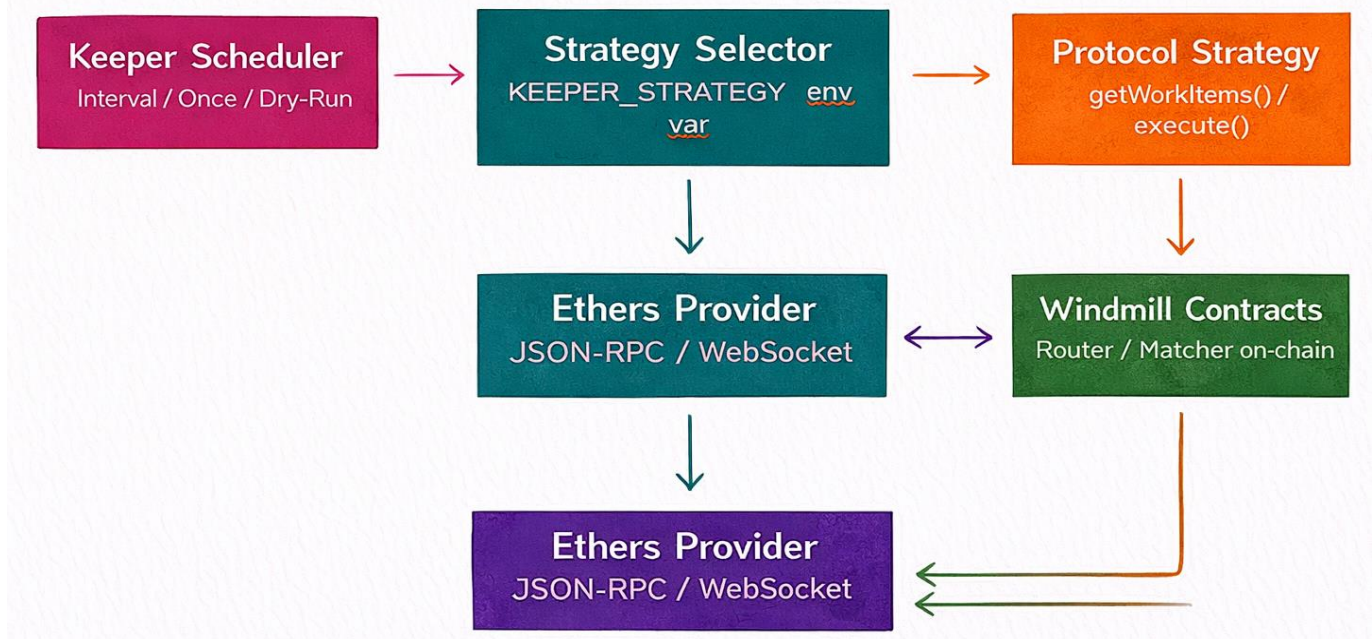
Transition	Trigger	Effect
PLACED → ACTIVE	<code>storeOrder()</code> called by Router	<code>isActive</code> = true, <code>amountFilled</code> = 0, all params immutable
ACTIVE → PARTIAL FILL	<code>updateFill()</code> called by Matcher	<code>amountFilled</code> increases, <code>isActive</code> stays true, price curve unchanged
PARTIAL FILL → ACTIVE	No action needed	Order continues accumulating fills — same state, same curve
ACTIVE → CLOSED (fill)	<code>deactivate()</code> by Matcher	<code>isActive</code> = false when <code>amountFilled</code> = <code>amountIn</code>
ACTIVE → CLOSED (cancel)	<code>cancelOrder()</code> via Router	<code>isActive</code> = false; unfilled <code>tokenIn</code> returned to maker
ACTIVE → CLOSED (expiry)	<code>isExpired()</code> check in Router	<code>isActive</code> = false when <code>block.timestamp</code> > <code>expiry</code> ; no token movement
ACTIVE → CLOSED (boundary)	<code>isFillable()</code> check	Auto-cancel when price crosses floor or ceiling — <code>tokenIn</code> refunded

Partial Fill Example:



THE KEEPER SYSTEM

What it is, why it exists, and where it fits in the Windmill system



The Windmill-EVM-Keeper is an off-chain Node.js daemon that bridges the gap between price-curve intersection events and on-chain settlement. It is completely trustless, it can only call public contract functions that any wallet could call. the only advantage is automation and timing.

What It Is

A headless Node.js service that runs on a VPS or cloud instance. No UI. No database. Reads blockchain state and submits transactions only when economically necessary, when two order price curves have crossed.

Why It Exists

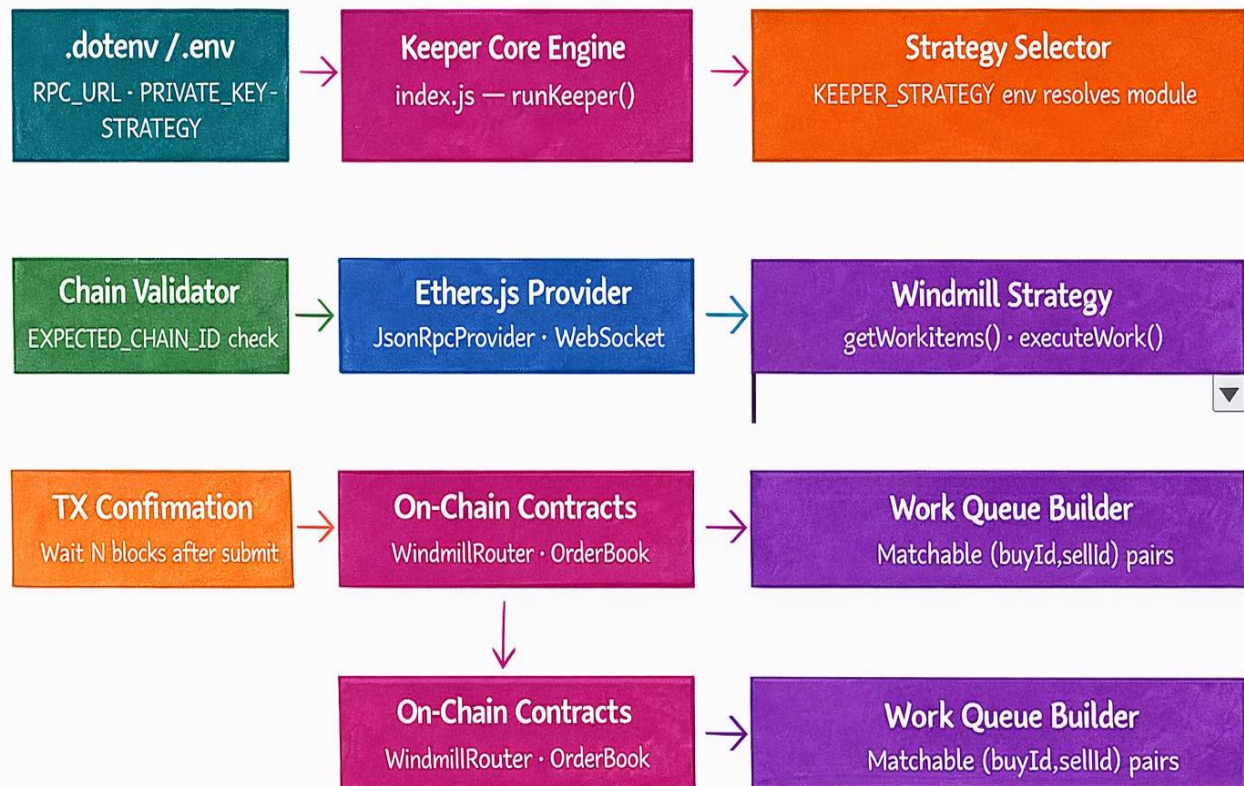
Smart contracts are passive and they only act when called. Without a keeper, two orders might intersect but never settle until a user manually triggers it. The Keeper automates this, ensuring markets clear continuously.

Why It Is Safe

The Keeper wallet has no special privileges. It calls `batchMatch()` which is publicly callable by any address. If the Keeper submits wrong data, the contract's `require()` checks reject the call and revert the transaction.

Property	Explanation
Permissionless	Anyone can call matchOrders() , keeper is just an automated version of this action
Trustless	No privileged role. Contract validates all inputs. Failed matches revert with no state change.
Configurable	Strategy, interval, batch size, dry-run mode , all controlled via environment variables
Gas Efficient	batchMatch() amortises base gas cost across N settlements per transaction
Graceful Failure	Error in one pair does not abort the batch , handled per-pair with catch blocks

Keeper — Full System Architecture



Component Responsibilities:

Module	What it does
index.js	Main starting file. Runs loop: load config → check chain → run strategy → wait → repeat. Handles errors so app don't crash.
config.js	Loads env variables. Checks all required values exist. Stops app if something missing.
providers.js	Creates RPC provider and wallet. Also checks correct chainId before use.
strategies/windmill.js	Runs main logic. Gets orders, checks matches, and calls batchMatch.
strategies/noop.js	Dummy strategy. Just logs, does nothing. Used for testing.
logger.js	Handles logs with time, level, and info. Used instead of console.log.

Keeper — Execution Cycle & Internal Flow



Folder structure (**expected**, yet to be discussed with mentor)

```
Windmill-EVM-Keeper/
├── src/
│   ├── index.js      ← Entry point: runKeeper() loop
│   ├── strategies/
│   │   ├── noop.js   ← Default no-op strategy (template)
│   │   └── windmill.js ← Windmill-specific: scans & matches orders
│   ├── providers.js  ← Ethers.js provider + signer factory
│   ├── config.js     ← dotenv loader + env validation
│   └── logger.js     ← Structured log output
├── test/             ← Unit tests (Mocha/Chai)
├── .github/workflows/ ← CI + security audit pipelines
└── .env.example      ← Config template with all variables
```

Keeper Internal Workflow:

Step	Action	Detail
1. Start	Load .env config	RPC_URL, PRIVATE_KEY, CONTRACT_ADDRESS, KEEPER_STRATEGY, EXPECTED_CHAIN_ID
2. Validate	Chain ID check	Confirms keeper is connected to the correct network before any action
3. Poll	Call getActiveOrders()	Reads all active order IDs from WindmillOrderBook via ethers.js provider
4. Detect	isMatchable() for each pair	For each buy/sell combination, calls PriceLib.isMatchable() view function
5. Queue	Build work item list	Collects all matchable (buyId, sellId) pairs into an action queue
6. Dry-run check	DRY_RUN flag	If true, logs actions but does NOT submit any transaction
7. Execute	batchMatch(buyIds[], sellIds[])	Submits batch transaction with up to MAX_ACTIONS_PER_CYCLE pairs
8. Confirm	Wait TX_CONFIRMATIONS blocks	Waits for on-chain confirmation before marking cycle complete
9. Repeat	Sleep KEEPER_INTERVAL_MS	Default 15 seconds between cycles; configurable per deployment

Keeper Configuration Parameters:

Parameter	Type/Default	Purpose
KEEPER_STRATEGY	Strategy module to load	Selects the protocol-specific logic (Windmill order matching strategy)
RPC_URL	JSON-RPC endpoint	Base Mainnet / Sepolia / Polygon / BNB — any EVM-compatible RPC
EXPECTED_CHAIN_ID	Safety check integer	Prevents keeper from running on wrong network (e.g. mainnet vs testnet)
PRIVATE_KEY	Keeper signer wallet	Funded wallet that pays gas for match transactions
MAX_ACTIONS_PER_CYCLE	Integer (default 25)	Caps batch size to avoid exceeding block gas limit
DRY_RUN	Boolean (default false)	Log-only mode — detects matches but submits no transactions
KEEPER_INTERVAL_MS	Milliseconds (default 15000)	Polling frequency — 15s matches approximate Ethereum block time
TX_CONFIRMATIONS	Integer (default 1)	Blocks to wait after tx submit before considering cycle complete

Keeper's Quadratic Complexity Problem

1 · THE PROBLEM: $O(N^2)$ Pair Scanning in the Keeper

Right now the keeper gets all active orders, then checks every buy with every sell. This means if orders increase, checks grow very fast (buy $\times$ sell). Because of this, it becomes very slow and costly when the order book gets big.

Example:

Active Orders (N)	Buy Orders (B)	Sell Orders (S)	Naïve Checks B×S	Optimised $O(N \log N)$
50	25	25	625	282
100	50	50	2,500	664
200	100	100	10,000	1,529
500	250	250	62,500	4,483
1,000	500	500	250,000	9,966

Keeper Pair-Check Complexity vs Order Book Depth

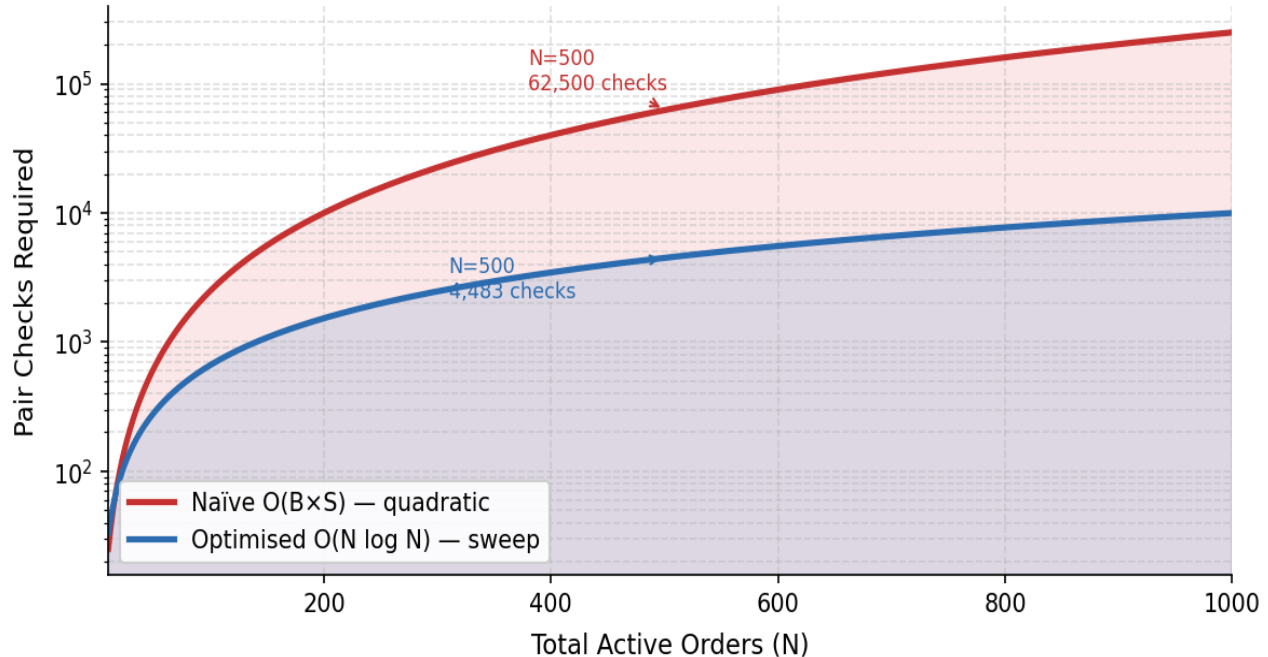


Figure 1: Log-scale growth of pair checks: quadratic blow-up (red) vs near-linear sweep (blue). Y-axis in log scale.

2 · ROOT CAUSE ANALYSIS

Root Cause: The Keeper has no ordering guarantee on the returned order list. Without sorting by current price, it cannot prune the search space, every buy must be tested against every sell.

3 · PROPOSED SOLUTION: Price-Sorted Two-Pointer Sweep Algorithm

Sort buy orders from high price to low, and sell orders from low to high. Then use two pointers and compare them. When prices match, move both pointers forward. Stop when highest buy price is less than lowest sell price. Send matched pairs using batchMatch(), no contract changes needed.

Step	Action	Detail
1	Fetch + Price-Stamp	Call getActiveOrders(). For each order, call PriceLib.currentPrice(order, block.timestamp) — one view call, $O(N)$.
2	Dual Sort	Sort buy array descending by current price; sort sell array ascending by current price. Cost: $O(N \log N)$ each.
3	Two-Pointer Init	Set $i = 0$ (highest buy), $j = 0$ (lowest sell). Both pointers start at the most likely match.
4	Sweep & Match	While $\text{buys}[i].\text{price} \geq \text{sells}[j].\text{price} \rightarrow$ add (buyId, sellId) to work queue, advance $i++$, $j++$.
5	Early Exit	As soon as $\text{buys}[i].\text{price} < \text{sells}[j].\text{price} \rightarrow$ STOP. Remaining pairs guaranteed non-matchable.
6	Batch Submit	Pass collected pairs to batchMatch(buyIds[], sellIds[]). Existing contract logic unchanged.

Table 2: Six-step two-pointer sweep replaces the nested loop. On-chain contracts are untouched.

Two-Pointer Sweep Algorithm — Visual Walkthrough

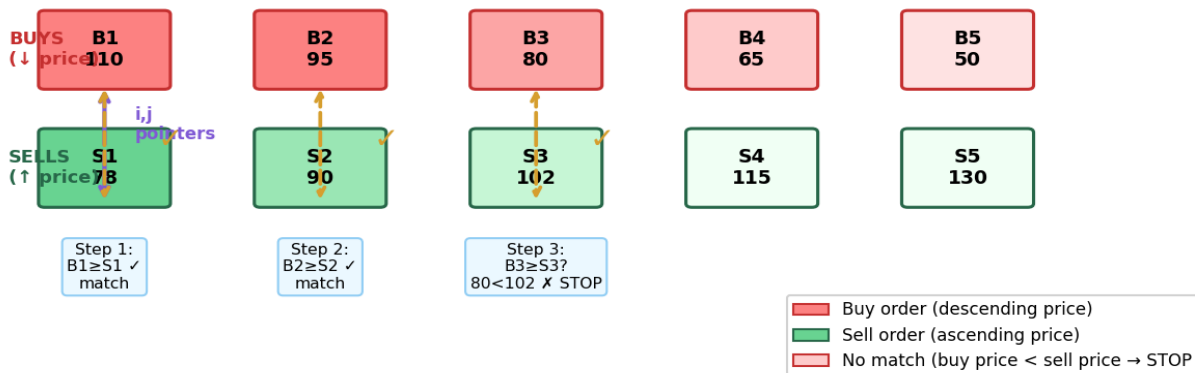


Figure 2: Two-pointer walkthrough: sorted arrays enable early termination — only 3 comparisons needed to find 2 matches and confirm the rest cannot match.

Pseudocode (strategies/windmill.js)

```
const now = await provider.getBlockNumber();
const orders = await orderBook.getActiveOrders();

// 1. Price-stamp all orders via multicall (one round-trip)
const priced = await multicallPrice(orders, now);

// 2. Dual sort –  $O(N \log N)$ 
const buys = priced.filter(o=>o.isBuy).sort((a,b)=>b.price-a.price);
const sells = priced.filter(o=>!o.isBuy).sort((a,b)=>a.price-b.price);

// 3. Two-pointer sweep –  $O(N)$ 
const pairs = []; let i=0, j=0;
while (i<buys.length && j<sells.length) {
  if (buys[i].price >= sells[j].price) {
    pairs.push([buys[i].id, sells[j].id]); i++; j++;
  } else break; // ← early exit, remaining pairs cannot match
}

// 4. Batch submit – unchanged
await router.batchMatch(pairs.map(p=>p[0]), pairs.map(p=>p[1]));
```

4 · COMPLEXITY & GAS ANALYSIS

Metric	Naïve Approach	Proposed Approach	Improvement
Time Complexity	$O(B \times S) \approx O(N^2/4)$	$O(N \log N)$	~25× at N=1,000
isMatchable() Calls (N=500)	62,500 calls	4,483 calls	13.9× fewer
RPC Round-trips	N^2 view calls	1 multicall + $O(N \log N)$ sort	near-constant
Gas (view calls, N=1000)	~50M gas units	~2M gas units	~25× reduction
On-chain Contract Changes	N/A	None required	✓ Zero risk

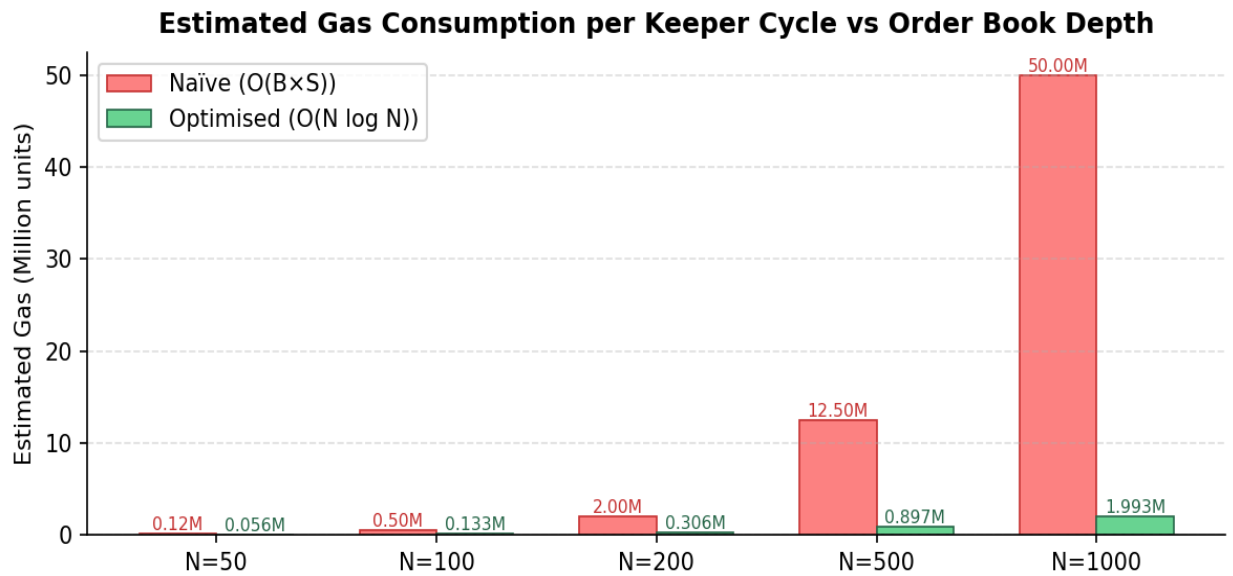


Figure 3: Estimated keeper gas consumption per cycle. At $N=1,000$ active orders the naïve approach consumes $\sim 50M$ gas vs $\sim 2M$ for the sweep — a $25\times$ difference.

Mathematical Derivation — Why Sorting Enables Early Exit

Given: buys sorted descending by price $\rightarrow \text{buys}[0].\text{price} \geq \text{buys}[1].\text{price} \geq \dots \geq \text{buys}[B-1].\text{price}$

Given: sells sorted ascending by price $\rightarrow \text{sells}[0].\text{price} \leq \text{sells}[1].\text{price} \leq \dots \leq \text{sells}[S-1].\text{price}$

Proof of early exit: If $\text{buys}[i].\text{price} < \text{sells}[j].\text{price}$, then:

$$\forall k \geq i : \text{buys}[k].\text{price} \leq \text{buys}[i].\text{price} < \text{sells}[j].\text{price} \leq \text{sells}[j].\text{price}$$

$$\forall l \geq j : \text{sells}[l].\text{price} \geq \text{sells}[j].\text{price} > \text{buys}[i].\text{price} \geq \text{buys}[k].\text{price}$$

$\therefore$ No pair (k, l) with $k \geq i$, $l \geq j$ can be matchable. The sweep terminates in $O(M)$ additional work, where M is the number of matched pairs.

$$\text{Total: } O(N \log N + M) = O(N \log N).$$

Transaction Handling & Error Recovery



<u>Dry-Run Mode</u>	<u>Batch Cap</u>	<u>Interval Tuning</u>	<u>Chain Guard</u>
Set DRY_RUN=true. Keeper runs full cycle , loads config, fetches orders, detects matches but never submits any transaction. Logs what it would do.	MAX_ACTIONS_PER_ CYCLE caps how many pairs are submitted per tx. Prevents block gas limit issues on high-volume markets. Default: 25.	KEEPER_INTERVAL_MS sets sleep between cycles. Default 15000ms matches Ethereum block time. Set lower for faster chains (Base: ~2s).	EXPECTED_CHAIN_ID aborts startup on mismatch. Prevents accidental mainnet execution while configured for testnet or vice versa.

Keeper — Configuration

Variable	Default	Purpose
KEEPER_STRATEGY	noop / windmill	Strategy module to load. 'windmill' activates the Windmill order-matching logic.
RPC_URL	https://...	JSON-RPC endpoint. Base Mainnet: mainnet.base.org · Sepolia: sepolia.base.org
EXPECTED_CHAIN_ID	Integer	Safety guard. Keeper aborts if provider's chainId doesn't match this value.
PRIVATE_KEY	0x...hex...	Keeper signer wallet private key. Must be funded with native gas token.
CONTRACT_ADDRESS	0x...hex...	Address of WindmillRouter deployed on target chain.
MAX_ACTIONS_PER_CYCLE	25 (default)	Maximum order pairs settled per transaction. Caps batch gas usage.
KEEPER_INTERVAL_MS	15000 (default)	Milliseconds between polling cycles. Match to chain block time for optimal coverage.
TX_CONFIRMATIONS	1 (default)	Blocks to await after tx submit. Increase for chains with frequent reorgs.
DRY_RUN	false (default)	When true: full cycle runs but zero transactions submitted. Safe for testing.
LOG_LEVEL	info (default)	Log verbosity: debug / info / warn / error.

EVENT SYSTEM & INDEXING



Event Signature	Emitter
OrderPlaced (orderId, maker, tokenIn, tokenOut, amountIn, startPrice, priceSlope, ts)	Emitted by Router.placeOrder()
OrderMatched (buyId, sellId, fillAmount, clearPrice, ts)	Emitted by Matcher._executeFill()
OrderCancelled (orderId, maker, ts)	Emitted by Router.cancelOrder()
OrderExpired (orderId, finalPrice, ts)	Emitted by Router when expiry is reached
PartialFill (orderId, filled, remaining)	Emitted by Matcher after every partial settlement
FeeDeposited (token, amount, ts)	Emitted by FeeVault.depositFee()

SECURITY ARCHITECTURE



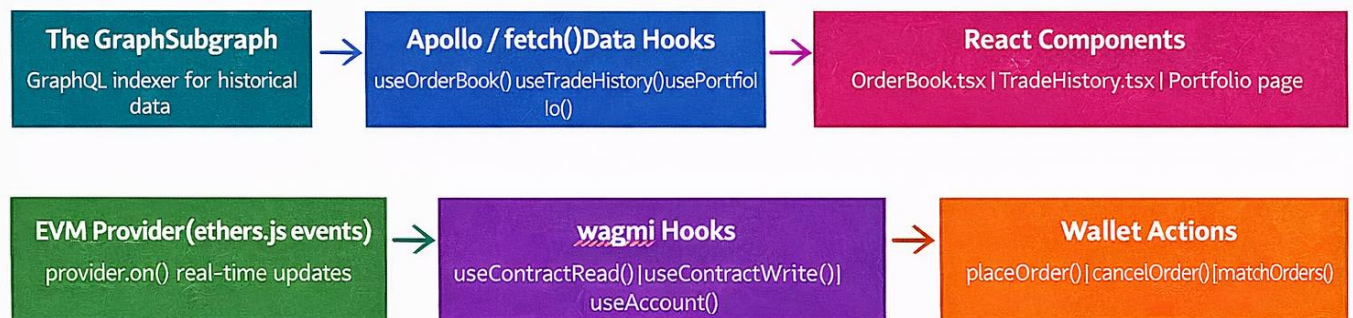
Risk	Problem	Solution
Reentrancy	Token tries to attack during transfer	Use safe pattern (CEI) and ReentrancyGuard
Access Control	Wrong user tries to change order	Only Router/Matcher can update orders
Price Overflow	Price becomes too big and breaks	Limit price range and use safe math
MEV Front-Running	Someone else tries to steal trade	Anyone can call match, no single keeper
Stale Orders	Old orders stay and waste gas	Expired orders are skipped early
Wrong Chain	Keeper runs on wrong network	Check chain ID before starting

Frontend — Overview & Architecture

Next.js 14 App Router · wagmi · RainbowKit · Tailwind CSS · The Graph

The Windmill WebUI is a stateless single-page application. It holds no privileged state — all data comes from on-chain events (via The Graph) or live contract reads (via wagmi). Every user interaction that changes state is a signed blockchain transaction.

Data Flow Architecture:



Tech Stack

Framework	Next.js 14 — App Router + Server Components
Language	TypeScript — strict mode throughout
Wallet	RainbowKit 2.x — MetaMask, WalletConnect, Coinbase
Chain Hooks	wagmi v2 — useContractRead, useContractWrite
Styling	Tailwind CSS — utility-first, dark-mode ready
Charts	Recharts — price curve visualisation
Indexer	The Graph — GraphQL subgraph for order history
Build	Vercel / IPFS — static export deployment

Design Principles

Stateless UI	No backend server. All state is on-chain or in wagmi/React Query cache.
Real-Time	provider.on() event subscriptions update order book instantly.
Trustless	No API key, no server wallet. User's own wallet signs all txns.
Progressive	Works read-only without wallet. Connect only needed to place orders.
Responsive	Tailwind breakpoints: mobile order placement, desktop full layout.
Accessible	ARIA labels, keyboard navigation, colour contrast AA compliance.

Expected repo structure

```
app/ (Next.js 14 App Router)
├── layout.tsx      ← Root layout: Providers, Header, Footer
├── Providers.tsx   ← WagmiProvider + RainbowKitProvider + QueryClient
├── page.tsx        ← Landing / Home — token pair selector
├── trade/[pair]/
│   ├── page.tsx    ← Main trading view
│   ├── OrderBook.tsx ← Live bid/ask ladder with price curves
│   ├── PlaceOrderPanel.tsx ← Order creation form + live preview
│   └── TradeHistory.tsx ← Settled trade feed
├── portfolio/
│   ├── page.tsx     ← User portfolio dashboard
│   ├── OpenOrders.tsx ← Active orders + cancel button
│   └── BalanceCard.tsx ← Vault token balances
├── analytics/
│   └── PriceCurveChart.tsx ← Intersection visualiser (Recharts)
└── components/
    ├── Header.tsx      ← Nav + wallet connect button
    ├── ConnectButton.tsx ← RainbowKit custom connect
    ├── PairSelector.tsx ← Factory.allPairs() dropdown
    ├── OrderCard.tsx    ← Reusable order row component
    ├── PriceTag.tsx     ← Live currentPrice() display
    └── StatusBadge.tsx  ← Order status: Open/Partial/Filled
```

State management:

WAGMI STORE

account • chainId
isConnected • balance

REACT QUERY CACHE

active orders
trade history

LOCAL UI STATE

form inputs • preview
modal open • filters

Page Layout & UI Components

<u>ORDER BOOK</u> Live bid/ask ladder Animated price rows Color-coded depth Time-to-intersect badge	<u>PLACE ORDER PANEL</u> Start Price field · Price Slope field Floor / Ceiling · Amount In Order Type toggle · Live Curve Preview Approve + placeOrder() buttons	<u>TRADE HISTORY</u> Real-time settlement feed Clear price + gas cost Pair filter · CSV export
<u>PRICE CURVE CHART (analytics/)</u> P_buy(t) vs P_sell(t) visualised · intersection zone highlighted	<u>PORTFOLIO DASHBOARD (portfolio/)</u> Open orders · Fill progress bars · Vault balances · P&L	FOOTER · Network Status Badge · Latest Block Number · Docs Link · AOSSIE / Discord

Component Reference

Component	What it does
Header.tsx	Top navigation bar. Has pair selector, wallet connect button, and links. Always visible.
ConnectButton.tsx	Wallet button. Shows short address + avatar. Opens wallet popup on click.
PairSelector.tsx	Dropdown of trading pairs. Gets data from Factory. Goes to selected pair page.
OrderBook.tsx	Shows buy and sell orders. Gets data from The Graph. Displays price, amount, time, etc.
PlaceOrderPanel.tsx	Form to place order. Enter price, amount, etc. Shows live curve. Sends transaction.
PriceCurveChart.tsx	Graph of buy/sell price curves. Shows where they meet. Updates every 10 sec.
TradeHistory.tsx	List of past trades. Shows amount, price, time, gas. Can filter by pair.
OpenOrders.tsx	User's current orders. Shows fill progress. Can cancel orders.
BalanceCard.tsx	Shows user token balances in FeeVault. Only view, no action.
OrderCard.tsx	Single order display. Shows price, status, and details.
PriceTag.tsx	Shows current price of order. Updates every block.
StatusBadge.tsx	Small label showing status: Open, Partial, Filled, Cancelled.

Frontend — Hooks, Indexing & Wallet Integration

wagmi contract hooks, The Graph subgraph, and the full wallet interaction flow

Wallet Interaction Flow:



wagmi Read Hooks

<code>useContractRead</code>	PriceLib.currentPrice() per order — live refresh every block
<code>useContractRead</code>	Factory.allPairs() — populates PairSelector dropdown
<code>useContractRead</code>	OrderBook.getActiveOrders() — used by manual match UI
<code>useBalance</code>	Native token balance of connected wallet
<code>useAccount</code>	address, chainId, isConnected status

wagmi Write Hooks

<code>useContractWrite</code>	ERC-20.approve(router, MAX_UINT256)
<code>useContractWrite</code>	Router.placeOrder(params) — creates new order
<code>useContractWrite</code>	Router.cancelOrder(orderId) — cancels active order
<code>useContractWrite</code>	Router.matchOrders(buyId,sellId) — manual settle
<code>useWaitForTransactionReceipt</code>	Polls until tx confirmed; triggers refetch

The Graph Subgraph — Event Handlers

Event	What happens / Used for
OrderPlaced	Adds new order. Marks it active. Used to show orders in OrderBook.
OrderMatched	Creates trade record (buy, sell, amount, price, time). Used in TradeHistory and charts.
PartialFill	Updates order fill amount. Marks it partial. Updates progress bar.
OrderCancelled	Marks order cancelled and inactive. Removes it from OrderBook.
OrderExpired	Marks order expired and inactive. Shown in history/portfolio.
FeeDeposited	Records fees collected. Used to show total revenue in dashboard.

Full System Integration

How the three repos — Contracts, Keeper, and WebUI — work together end-to-end

End-to-End Order Flow



DEPLOYMENT TOPOLOGY

The three repositories deploy to distinct infrastructure. Smart contracts live on-chain; the Keeper runs as a VPS daemon; the WebUI hosts on IPFS/Vercel. All three are independent and stateless relative to each other.

TESTNET (Sepolia)	MAINNET (EVM L1/L2)	OFF-CHAIN SERVICES
WindmillFactory.sol WindmillRouter.sol WindmillOrderBook.sol WindmillMatcher.sol WindmillPriceLib.sol WindmillFeeVault.sol	Same contract set deployed via Factory per trading pair Base Mainnet (Chain 8453) Polygon PoS (Chain 137) BNB Chain (Chain 56)	Keeper Bot (Node.js, VPS) The Graph Subgraph (indexer) IPFS / Vercel (WebUI hosting) Alchemy / Infura (RPC provider)

Deployment Sequence:

#	Step	Dependencies
1	Deploy WindmillPriceLib	Library has no constructor dependencies — deploy first
2	Deploy WindmillFeeVault	Constructor takes treasury address and initial fee rate
3	Deploy WindmillFactory	Constructor takes FeeVault address and PriceLib address
4	Create trading pair	Call <code>Factory.createPair(tokenA, tokenB)</code> — deploys Router + OrderBook + Matcher
5	Configure Keeper	Set <code>CONTRACT_ADDRESS</code> , <code>RPC_URL</code> , <code>EXPECTED_CHAIN_ID</code> in <code>.env</code> ; fund keeper wallet
6	Start Keeper Bot	<code>npm run start</code> — begins polling cycle on <code>KEEPER_INTERVAL_MS</code>
7	Deploy Subgraph	Point The Graph manifest to Router address; define event handlers
8	Deploy WebUI	Set RPC and contract addresses in <code>Next.js</code> env; deploy to Vercel/IPFS
9	Verify Contracts	Run Hardhat verification against Etherscan/Blockscout for each contract
10	Security Audit	Run Slither and Mythril; submit findings to security review before mainnet

WEEKLY EXECUTION TIMELINE

Phase	Weeks	Theme	Deliverables
Phase 0	Bonding	Onboarding	Final design note reviewed by mentors. Order struct finalized. Repo structure agreed.
Phase 1	W1 – W3	Core Contracts	WindmillPriceLib.sol complete + fuzz tested. WindmillOrderBook.sol with placeOrder / cancelOrder. Deploy to Sepolia. Unit tests > 90%.
Phase 2	W4 – W6	Matching Engine	WindmillMatcher.sol with tryMatch() + partial fills. WindmillFeeVault.sol. Integration tests end-to-end. Gas benchmarks documented.
Phase 3	W7 – W8 midterm	Router & Indexer	WindmillRouter.sol. Off-chain Node.js indexer. WindmillFactory.sol for multi-pair support. Full test suite passing.
Phase 4	W9 – W11	Frontend	All 5 pages built. Price curve visualizer live. RainbowKit wallet integration. Testnet demo working.
Phase 5	W12	Polish	Gas optimization pass. Security review checklist. Full README + deployment guide. Demo video. Final submission.

Karma Protocol

A Self-Sovereign Prediction Market Oracle

1 PITCH:

Before diving into the project, let's first find out what's wrong or what is missing in current market price fetching scenario:

See in current system, to fetch price of crypto assets we rely on some external sources and we don't know how exactly that source is getting that data and providing to us. This can be blindly trusting on someone or external Oracle.

Now question arises what's wrong in this if an external Oracle is working good and millions of people believing then why not we should?

Even if an oracle is reliable and widely trusted, it still introduces a **central point of trust and failure**. But what if I say we can **eliminate this single point trust risk** along with getting the best price then?

Yes this is possible in the world of Web3, this can be achieved easily.

INTRODUCING **KARMA PROTOCOL**:

Karma Protocol is a **fully on-chain, trustless price discovery system** that eliminates the need for external oracles entirely.

Instead of relying on a fixed set of data providers, it uses the fact that how people react to knowing the price of some asset, and how they use their money. In simple words it finds out the people's action based on these actions price is predicted (more generally calculated). Let's see how (in quick).

Karma introduces two tokens **bullCoin** and **bearCoin** which represent the direction a participant believes the price should move.

When a user enters the system, they deposit real capital and choose a position:

- bullCoin if they think the price is lower than it should be
- bearCoin if they think the price is higher than it should be

Now, if the current price in Karma is not aligned with the real market price, participants who know the correct value will take positions accordingly to profit from the mismatch.

Here comes the actual working as the bullCoins and bearCoins compete to get corresponding profits it actually reaches the Equilibrium position which tells the best price of that asset.

Thus we can say that, Karma doesn't fetch prices rather it **discovers them** by continuously adjusting based on market pressure until it reaches equilibrium with the broader market.

1.1 PROJECT VISION

The vision of this project is to reduce dependence on external oracles by introducing a trustless, on-chain price discovery mechanism for suitable use cases like prediction markets..

1.1 The Oracle Problem in Prediction Markets

Fate is currently a well-designed prediction market but it has one architectural weakness:

it depends entirely on an external oracle (Chainlink or similar) to determine prices. This is a live single point of failure. If the oracle delays, is manipulated, goes offline, or simply becomes expensive, the entire prediction market breaks.

Let's see how karma solves this:

Problem	Why It Matters	Karma's Answer
External oracle dependency	If Chainlink depreciates a feed, the market freezes	Replace with participant-submitted, on-chain oracle
Oracle manipulation	A well-funded actor can move a Chainlink feed briefly to profit on Fate positions	Weight submissions by neutrality and biased actors have near-zero influence
Cost and latency	Each oracle read is a cross-contract call with gas overhead	getPrice() is a pure view on Karma's own state , zero external gas
Centralisation risk	All Chainlink price feeds are controlled by Chainlink node operators	Anyone with bullCoin or bearCoin can submit ,permissionless
No price discovery	External oracle brings price from outside; market participants have no role	Karma encodes market participants' own consensus into the price feed

1.2 The Naive Participant Oracle — Why It Fails

- A user holding only bullCoin will always try to push the price **higher** to make profit.
- A user holding only bearCoin will always try to push the price **lower** for the same reason.
- If we just average all submitted prices, the one with more tokens can **easily manipulate the result**.
- Fake (sybil) accounts with small balances can **spam submissions** and disturb the price at very low cost.

Why Karma Wins

Karma solves this by not restricting participation, but by **automatically reducing the influence of biased users and increasing the weight of neutral ones**.we will see it in upcoming sections(1.4):

Correct understanding: Both serve different needs

Chainlink = **reliable service (with trust)**

Karma = **trustless system (with incentives)**

Karma aims to replace oracles where trustless price discovery is possible and not everywhere.

1.4 The Karma Insight — Neutrality as Weight

Karma borrows GluonReactor.sol's two-token weight function and applies it to oracle price submissions. The more balanced a submitter's bullCoin/bearCoin position, the higher their oracle weight. The formula is here below:

```
// Weight function (adapted from GluonReactor.sol)
// b = bullCoin balance of submitter
// r = bearCoin balance of submitter
// ε = small constant to prevent division-by-zero

weight(submitter) = min(b, r) / (b + r + ε)

// Examples:
// b=500, r=500 → weight = 500/1000 = 0.500 (maximum influence)
// b=900, r=100 → weight = 100/1000 = 0.100 (mostly ignored)
// b=999, r=001 → weight = 001/1000 = 0.001 (nearly zero)
// b=100, r=000 → weight = 0/100 = 0.000 (zero influence)
```

This is then combined with Orb's exponential decay so that stale submissions automatically lose influence over time, regardless of weight: here we can see:

```
// Decayed weighted price average (adapted from OrbOracle-Solidity)

P_oracle = Σ [ weight(i) × decay(t_i) × price(i) ] / Σ [ weight(i) × decay(t_i) ]

decay(t) = e^( -λ × Δt )
λ = decay parameter (configurable, e.g. 0.001 per second)
Δt = seconds since that submission was made
// Higher λ = faster staleness. Lower λ = longer memory.
```

1.5 The “**DO’S**” AND “**DON’TS**” define the **intended scope and boundaries** of Karma Protocol, ensuring

Do’s (What Karma Does)	Don’ts (What Karma Does NOT Do)
Uses a fully on-chain system to determine price	Does not replace services like Chainlink with SLA guarantees
Removes dependency on a single external oracle	Does not guarantee continuous uptime or availability
Relies on participant actions and economic incentives	Needs active participation; cannot work without users
Allows anyone to participate (permissionless)	Does not fetch real-world external data (e.g., sports results)
Discovers price from market behavior	Only works based on participant consensus
Uses bull and bear positions to signal direction	Does not automatically balance user positions
Adjusts price based on capital flow (bull vs bear)	Users must manage their own positions manually
Rewards neutral participants with higher influence	Does not include governance for parameters like λ (in this scope)
Reduces influence of biased participants	Some parameters are fixed initially and updated manually
Works without trusted intermediaries	Can be manipulated if one entity controls most tokens
Uses real capital to ensure honest participation	Does not eliminate need for external information
Continuously updates price toward market equilibrium	Does not guarantee perfectly accurate price at all times
Enables decentralized price discovery	Not suitable for all use cases requiring strict reliability

2. PROJECT SCOPE AND DELIVERABLES

Exactly what gets built, and what is explicitly out of scope

2.1 Core Deliverables

Component	File / Module	Description	Technology
Karma.sol	contracts/Karma.sol	Main contract: extends Fate pool, replaces IOracle dependency, routes price through on-chain weighted average	Solidity 0.8.x, Foundry
WeightLib.sol	contracts/lib/WeightLib.sol	Extracted and adapted from GluonReactor.sol — pure library computing $\min(b,r)/(b+r+\epsilon)$ with fixed-point arithmetic	Solidity library
PriceAverager.sol	contracts/lib/PriceAverager.sol	Adapted from OrbOracle: running weighted decayed average. Accepts weight as parameter instead of raw token balance	Solidity library
IKarmaOracle.sol	contracts/interfaces/IKarmaOracle.sol	Chainlink-compatible oracle interface. External protocols can integrate without code changes	Solidity interface
Karma.t.sol	test/Karma.t.sol	Comprehensive unit + fuzz test suite: weight edge cases, averaging precision, replay attacks, flash loan simulation	Foundry / forge test
Price Submission UI	src/components/PriceSubmit.tsx	Frontend: input price, shows real-time weight preview before submit, displays recent submissions with weights	Vite + React 18
Oracle Dashboard	src/components/OracleDash.tsx	Current oracle price, time since last update, weight distribution chart, active submitter count	Vite + React
My Position Panel	src/components/MyPosition.tsx	Wallet-specific: bullCoin/bearCoin balances, computed weight, historical weight chart	wagmi, viem
Submitter Leaderboard	src/components/Leaderboard.tsx	Ranked list of top submitters by weight; submission frequency; price accuracy vs external market	Vite + React
Karma Poster	poster/index.ts	Off-chain cron service: fetches external price, submits to all 5 chains, manages balanced position, handles retries	Node.js 20 LTS, TypeScript
Deploy Scripts	script/Deploy.s.sol	Foundry deployment scripts for all 5 chains with constructor parameter configuration	Foundry forge script
CI/CD Pipeline	.github/workflows/	forge build + forge test on every PR; Slither analysis gating; auto-deploy to Vercel on main merge	GitHub Actions
NatSpec Docs	docs/	NatSpec for every public function; architecture diagrams; integration guide for external oracle consumers	Markdown, Mermaid

2.2 Core Source Repositories

Repository	Role in Karma	What Studied
Fate-Solidity	Starting point — prediction market base, bullCoin/bearCoin minting, market pool logic	FatePool.sol, oracle interface IOracle.sol, token minting flow
OrbOracle-Solidity	Decayed price averaging — exponential TWAP that Karma adapts for two-token weighting	submitPrice(), the weighted moving average update, lambda decay parameter
Gluon-Solidity	Two-token weight function — the mathematical heart of manipulation resistance	computeWeight() or equivalent; the $\min(b,r)/(b+r+\epsilon)$ formula
OrbOracle-Poster	Off-chain poster script template — Node.js price feed submitter Karma will adapt	Submission loop, gas management, nonce handling, multi-chain .env config
Fate-EVM-Frontend	Frontend starting point — Karma extends this with oracle UI components	Component tree, where IOracle price is consumed, wagmi hooks used

2.3 Karma vs Existing Oracle Solutions , Let's see Full Comparison:

Feature	Karma	Chainlink	Orb	Fate (before Karma)	UMA
External dependency	None — fully self-contained	Yes — Chainlink node operators	Partial — needs token holders	Yes — Chainlink	Yes — DVM voters
Two-token neutrality weight	Core mechanism				
Exponential decay TWAP	From Orb	(heartbeat model)			
Chainlink-compatible interface	Drop-in replacement	—			
Permissionless submission	Any holder	Whitelisted nodes	Partial		Partial
Flash loan resistance	Built-in via balance-at-submission	Varies	Partial	N/A	N/A
Sybil resistance	Capital cost	(different model)	Partial	N/A	Partial
Multi-chain (5 EVM)	ETC/ETH/Polygon/Base/BSC	Partial	Partial	Partial	Partial
Self-referential oracle	Market IS the oracle				
Gas cost for price read	Zero (view function)	~21k gas (cross-contract)	Zero	~21k gas	High

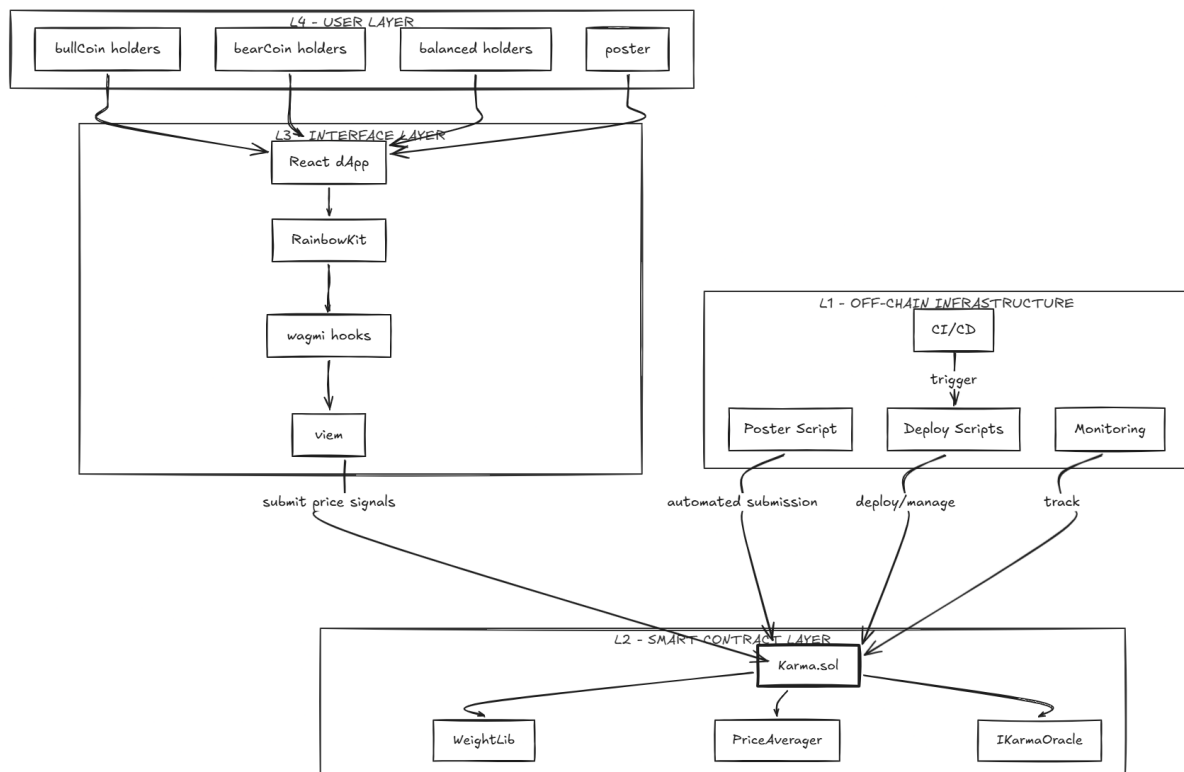
3 SYSTEM ARCHITECTURE AND DESIGN

How the four layers interact — top to bottom

3.1 Four-Layer Architecture Overview

Layer	Who / What	Responsibility	Key Technologies
L4 — User Layer	bullCoin holders, bearCoin holders, balanced holders, poster script	Submit price observations to the oracle; hold and manage positions	EVM wallets, MetaMask, WalletConnect
L3 — Interface Layer	Vite+React dApp, RainbowKit wallet modal, wagmi hooks, viem	Display oracle state, enable price submission, show weight previews, connect wallets	React 18, wagmi v2, viem v2, RainbowKit
L2 — Smart Contract Layer	Karma.sol, WeightLib.sol, PriceAverager.sol, IKarmaOracle.sol	Compute weights on-chain, maintain decayed average, expose oracle interface, enforce invariants	Solidity 0.8.x, Foundry, OpenZeppelin
L1 — Off-chain Infrastructure	Karma Poster (Node.js), GitHub Actions CI/CD, Foundry deployment scripts	Submit baseline price feed, run test pipelines, deploy and verify contracts on 5 chains	Node.js 20, TypeScript, GitHub Actions, forge deploy

HIGH LEVEL ARCHITECTURE DIAGRAM:



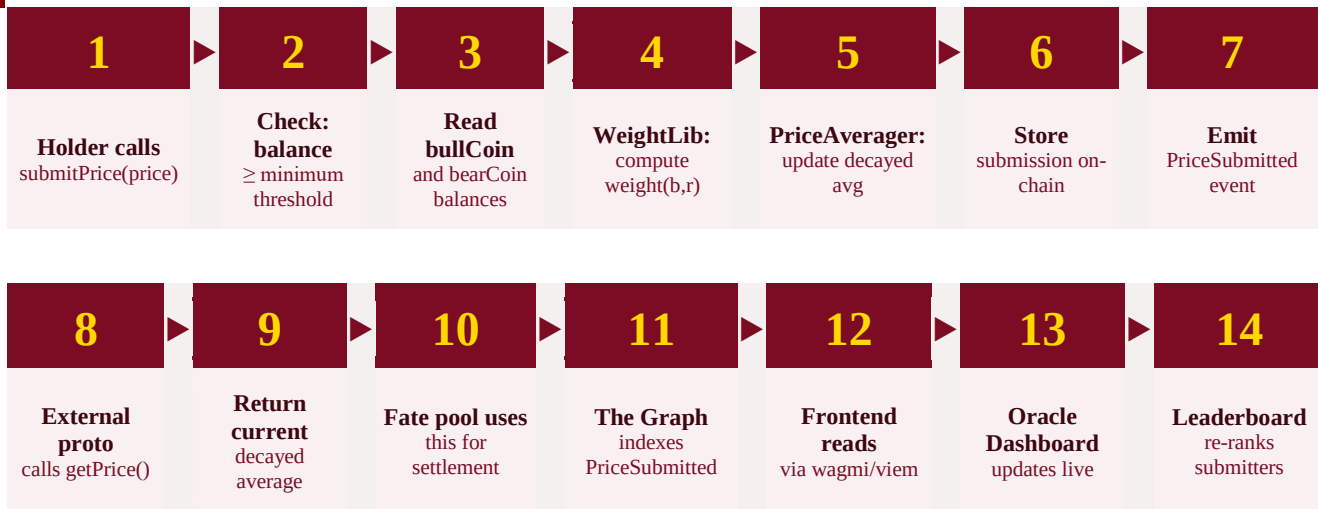
3.2 Contract Module Dependency Graph

The contract layer is modular by design. WeightLib and PriceAverager are pure libraries — they contain no state and can be audited in isolation. Karma.sol is the single stateful contract:

```
Karma.sol (stateful)|
├── inherits FatePool.sol      (prediction market logic)
├── uses lib WeightLib.sol     (stateless – weight computation)
├── uses lib PriceAverager.sol (stateless – decayed average)
└── implements IKarmaOracle.sol (Chainlink-compatible interface)

// Previously: FatePool.sol → IOracle.sol → Chainlink (external)
// After:      FatePool.sol → IKarmaOracle.sol → Karma.sol (self)
```

3.3 Price Submission Data Flow End to End



3.4 Why this architecture is suitable:

Design Principle

- * WeightLib and PriceAverager are pure libraries, so they don't store data and are easy to audit.
- * Only Karma.sol stores data, making it the main place where security matters.
- * IKarmaOracle follows Chainlink format, so it can be used easily by existing systems.
- * The frontend does not need a backend, it directly reads data from the blockchain.
- * All updates are taken from on-chain data and event logs, no custom server is required.

What This Enables

- * WeightLib can be tested separately without deploying the full contract.
- * PriceAverager can be tested quickly by mocking inputs.
- * Any system using Chainlink can switch to Karma by just changing the contract address.
- * The app still works even if the poster script is down.
- * Users can directly interact with the system through the frontend.

4. SMART CONTRACT LAYER (DEEP DIVE):

4.1 Starting Point — Studying Fate-Solidity

Before writing a single line of Karma, I have studied the Fate-Solidity codebase to understand exactly how the oracle price is currently consumed ,The key files are:

File	What it does now	What Karma changes / adds
PredictionPool.sol	Main contract. Creates bull and bear tokens, gets price from oracle, updates pool state	No change needed. Karma just replaces the oracle using updateOracle
IOracle.sol	Defines function to get latest price	Karma follows the same function so it fits directly
Coin.sol	ERC20 token used for both bull and bear, tracks balances	No change. Karma reads user balances to calculate influence
PredictionPoolFactory.sol	Creates new pools and sets valid oracle	Karma adds its own adapter to plug into this flow
ChainlinkOracle.sol	Gets price from Chainlink and formats it	Replaced by KarmaOracle which gives price from users
ChainlinkAdapterFactory.sol	Deploys and registers Chainlink oracle	Karma creates similar factory but deploys KarmaOracle

4.2 WeightLib.sol — The Mathematical Core

WeightLib is a pure Solidity library extracted and adapted from GluonReactor.sol. It contains no storage — it takes two balances and returns a weight in fixed-point uint256 (scaled by 1e18). Here is the full logic:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.19;

library WeightLib {
    uint256 constant EPSILON = 1e6; // prevents div-by-zero for tiny balances
    uint256 constant SCALE = 1e18; // fixed-point precision

    /// @notice Compute oracle submission weight from two-token balances.
    /// @dev    weight = min(bull, bear) / (bull + bear + EPSILON)
    /// @param bull bullCoin balance of submitter (wei units)
    /// @param bear bearCoin balance of submitter (wei units)
    /// @return w    weight in [0, 0.5 * SCALE], scaled by SCALE
    function computeWeight(
        uint256 bull,
        uint256 bear
    ) internal pure returns (uint256 w) {
        uint256 numerator = bull < bear ? bull : bear; // min(bull, bear)
        uint256 denominator = bull + bear + EPSILON;
        w = (numerator * SCALE) / denominator;
    }
}

return go(f, seed, [])
}
```

Edge cases and their behaviour:

Scenario	bull	bear	weight ($\times 1e18$)	Behaviour
Perfectly balanced	1000e18	1000e18	500000000000000000	Maximum influence (0.5)
Slightly imbalanced	600e18	400e18	400000000000000000	Good influence (0.4)
Heavily biased bull	900e18	100e18	100000000000000000	Low influence (0.1)
Extreme bias	999e18	1e18	~ 1000000000000000	Near zero (~ 0.001)
All bull, no bear	1000e18	0	~ 0	Zero weight (epsilon prevents div/0)
Both zero	0	0	0	Zero weight — cannot submit
Only dust balance	1	1	~ 499999500000000000	Full weight even for tiny balanced holdings

4.3 PriceAverager.sol — Decayed Weighted Average

Adapted from OrbOracle-Solidity's averaging logic. The key adaptation: instead of weighting by raw token balance, we accept a pre-computed weight from WeightLib. This separation of concerns keeps each library independently testable.

```
library PriceAverager {
    struct State {
        uint256 weightedPriceSum; //  $\Sigma (\text{weight} \times \text{decay} \times \text{price})$ 
        uint256 weightSum;        //  $\Sigma (\text{weight} \times \text{decay})$ 
        uint256 lastUpdateTime;    // block.timestamp of last submission
    }

    /// @notice Update running average with a new weighted, decayed submission.
    /// @param s      Current averager state (modified in place)
    /// @param price   Submitted price (scaled by 1e18)
    /// @param weight  Submitter weight from WeightLib (scaled by 1e18)
    /// @param lambda  Decay parameter (scaled by 1e18; e.g. 1e15 = 0.001/s)
    function update(State storage s, uint256 price, uint256 weight, uint256 lambda) internal {
        uint256 dt = block.timestamp - s.lastUpdateTime;
        uint256 decay = computeDecay(lambda, dt); //  $e^{(-\lambda * dt)}$  approx
        uint256 effectiveWeight = (weight * decay) / 1e18;
        s.weightedPriceSum = (s.weightedPriceSum * decay / 1e18) + effectiveWeight * price / 1e18;
        s.weightSum        = (s.weightSum * decay / 1e18) + effectiveWeight;
        s.lastUpdateTime    = block.timestamp;
    }

    function getPrice(State storage s) internal view returns (uint256) {
        require(s.weightSum > 0, "No submissions yet");
        return s.weightedPriceSum / s.weightSum;
    }
}

return go(f, seed, [])
```

Activ
Go to

How PriceAverager Works:

1. A new price is submitted along with its weight (from WeightLib).
2. The contract checks how much time has passed since the last update.
3. It applies decay, so older data becomes less important over time.
4. The new price is adjusted using its weight and added to the running average.
5. Both total weighted price and total weight are updated.
6. The latest timestamp is stored for the next update.
7. When price is requested, it returns: total weighted price $\div$ total weight

4.4 submitPrice() — The Central Transaction

This is the function every token holder calls. It is the entry point for all oracle data.

```
mapping(address => Submission) public submissions;
PriceAverager.State private _avgState;
uint256 public constant MIN_BALANCE = 1e15; // anti-dust threshold

event PriceSubmitted(address indexed submitter, uint256 price, uint256 weight, uint256 timestamp);

function submitPrice(uint256 price) external nonReentrant {
    uint256 bull = bullToken.balanceOf(msg.sender);
    uint256 bear = bearToken.balanceOf(msg.sender);

    // Minimum balance check – prevents dust-balance Sybil attacks
    require(bull + bear >= MIN_BALANCE, "Insufficient token balance");

    // Compute weight on-chain from current balances
    uint256 weight = WeightLib.computeWeight(bull, bear);

    // Update the decayed weighted average
    PriceAverager.update(_avgState, price, weight, lambda);

    // Record this submission for transparency / leaderboard
    submissions[msg.sender] = Submission({ price: price, weight: weight, timestamp: block.timestamp });

    emit PriceSubmitted(msg.sender, price, weight, block.timestamp);
}
```

Activate Wi-Fi

4.5 getPrice() — The Oracle Interface

```
interface IKarmaOracle {
    /// @return price Current decayed weighted average, scaled by 1e18
    function getPrice() external view returns (uint256 price);

    /// @notice Chainlink AggregatorV3 compatibility – drop-in replacement
    function latestRoundData() external view returns (
        uint80 roundId,
        int256 answer, // getPrice() cast to int256
        uint256 startedAt,
        uint256 updatedAt, // lastUpdateTime from averager state
        uint80 answeredInRound
    );
}
```

4.6 Security Analysis — Every Attack Vector

Attack	What attacker tries	Karma protection	Risk left
Flash loan	Borrow tokens, fake price	Balance checked in same tx	Almost none
Sybil spam	Many small wallets	Low balance = low weight	Very low
Big whale bias	One user holds many tokens	Unbalanced = low influence	Medium if majority
Old data reuse	Reuse old price	Decay reduces old data	None
Lambda misuse	Change decay to zero	Fixed at start	Low
Reentrancy	Attack during submit	nonReentrant used	None
Overflow	Break math with big values	Solidity safety checks	None
Low liquidity attack	Few users → easier to move price	Needs active users	Medium
Delayed updates	No one submits new prices	Old data decays slowly	Medium
Collusion attack	Group acts together to push price	Needs large balanced tokens	Medium
Front-running	See tx and submit before others	No strong direct protection	Low–Medium
Spam submissions	Send many tx to influence timing	Gas cost limits spam	Low
Time manipulation	Miner slightly shifts timestamp	Small impact due to decay	Low
Incorrect baseline (poster)	Poster submits wrong initial price	Market corrects over time	Low–Medium

- Karma reduces most attacks by using **economic incentives instead of trust**.
- The system becomes stronger as **more users participate and liquidity increases**.
- No single user can easily control the price unless they hold a very large share.
- Overall security comes from **balance, participation, and decentralization** rather than strict control.

5. FRONTEND LAYER

Extending components, oracle UI

5.1 Starting Point: Fate-EVM-Frontend Analysis

Component / File	Current Role	Karma Extension
pages/index.tsx or App.tsx	Entry point, routing	Add /oracle route for oracle dashboard
hooks/useFatePool.ts (or equiv)	wagmi hook reading FatePool contract	Extend to read submitPrice, getPrice, submissions mapping from Karma
components/PriceDisplay.tsx	Shows current oracle price from Chainlink	Replace Chainlink read with getPrice() from Karma contract
config/chains.ts	Chain config for wagmi	Add ETC, Polygon, Base, BSC if not present; add Karma contract addresses per chain
components/WalletConnect.tsx	RainbowKit wallet modal	Unchanged — already multi-wallet capable

5.2 New Components — What Gets Built

PriceSubmit.tsx

- User enters the price they see in market
- Shows their weight based on bull/bear balance
- Shows how much their price will affect the current price
- Submit button sends transaction and shows status
- Shows last 10 submissions with price and weight

OracleDash.tsx

- Shows current Karma price clearly
- Shows how long since last update
- Shows number of active users (last 24h)
- Shows how weight is split (balanced vs biased)
- Shows difference from external price

MyPosition.tsx

- Shows user's bull and bear token balance
- Shows current weight in real-time
- Shows how weight changed over time
- Suggests how to balance tokens for more influence

Leaderboard.tsx

- Top 20 users based on weight
- Shows how accurate their prices were
- Shows how often they submit
- Shows if they are balanced or biased

 FRONTEND (Next.js dApp)



Next.js Application



Wallet Layer

MetaMask
WalletConnect
RainbowKit Modal



Data Layer

wagmi (read/write)
viem (RPC calls)
Event logs



UI Components

PriceSubmit
OracleDash
MyPosition
Leaderboard



State / Hooks

useAccount
useContractRead
useContractWrite
useBalance

 WALLET CONNECTION FLOW

User → RainbowKit Modal



Connect Wallet



wagmi

Account Ready

 DATA FLOW (READ)



Smart Contract
(Karma.sol)

wagmi / viem
(read + events)

Next.js State / Hooks

UI Components
(Dashboard /
Leaderboard)

 DATA FLOW (WRITE)

User Input

PriceSubmit

wagmi write
submitPrice()



Smart Contract
(Karma.sol)

UI updates after
confirmation

5.3 Full Frontend Transaction Lifecycle

Step	Where	Action
1. Connect	RainbowKit modal	User opens wallet modal and connects their wallet (MetaMask, WalletConnect, etc.)
2. Read balances	wagmi hook	App fetches bull and bear token balances of the connected user
3. Compute weight	Client-side JS	App calculates user weight based on how balanced their tokens are
4. Compute impact	Client-side JS	App estimates how much the submitted price will affect current oracle price
5. Enter price	PriceSubmit.tsx	User enters market price and sees live weight and impact preview
6. Sign transaction	Wallet	Wallet prompts user to approve and sign the transaction
7. Broadcast	RPC node	Signed transaction is sent to blockchain through RPC
8. Wait confirmation	wagmi hook	App shows pending status and waits for transaction confirmation
9. Index event	The Graph	Submission event is stored and used for history and leaderboard
10. Update UI	wagmi refresh	UI updates automatically with new price, balances, and rankings

5.4 Backend-Free Architecture

All data comes directly from the blockchain:

- App reads price, balances, and data using wagmi/viem
- Past data is taken from event logs

There is **no backend server**.

Even if The Graph is down, the app still works using direct RPC calls.

5. OFF-CHAIN POSTER SCRIPT

The Node.js service that anchors the oracle to real-world prices

6.2 Starting Point: OrbOracle-Poster

OrbOracle-Poster Feature	Karma Poster Adaptation
Fetches price from external API (CoinGecko, Binance)	Unchanged — same price source logic
Calls submitPrice(price) on Orb contract	Calls submitPrice(price) on Karma contract on each configured chain
Uses single token balance to determine weight	Must track TWO balances: bullCoin and bearCoin; maintain balance between them
Single chain configuration	Extended to 5-chain config: ETC, ETH Mainnet, Polygon, Base, BSC
Retry logic on failed transactions	Unchanged and extended — exponential backoff per chain
.env config: RPC_URL, PRIVATE_KEY, CONTRACT_ADDRESS, INTERVAL	Extended: one RPC_URL + CONTRACT_ADDRESS per chain; all share same private key

6.3 Poster Script Architecture

```
// poster/index.ts – high-level structure
```

```
const CHAINS: ChainConfig[] = [  
  { name: 'ETC',      rpcUrl: process.env.ETC_RPC!,      contractAddress: process.env.ETC_KARMA! },  
  { name: 'ETH',      rpcUrl: process.env.ETH_RPC!,      contractAddress: process.env.ETH_KARMA! },  
  { name: 'Polygon',  rpcUrl: process.env.POLY_RPC!,      contractAddress: process.env.POLY_KARMA! },  
  { name: 'Base',     rpcUrl: process.env.BASE_RPC!,      contractAddress: process.env.BASE_KARMA! },  
  { name: 'BSC',      rpcUrl: process.env.BSC_RPC!,      contractAddress: process.env.BSC_KARMA! },  
];
```

```
async function runCycle() {  
  const price = await fetchExternalPrice(); // CoinGecko or Binance API  
  await Promise.allSettled(CHAINS.map(chain => submitToChain(chain, price)));  
}
```

```
async function submitToChain(chain: ChainConfig, price: bigint) {  
  // 1. Check our bull/bear balance on this chain  
  // 2. If imbalanced, log a warning (rebalancing is manual in this scope)  
  // 3. Estimate gas, set nonce, call submitPrice(price)  
  // 4. Wait for confirmation with timeout  
  // 5. On failure: exponential backoff, retry up to 3 times  
  // 6. Log result to structured JSON log  
}
```

```
setInterval(runCycle, INTERVAL_MS); // default: 300_000 (5 minutes)
```

6.4 Key Difference : Dual Balance

Karma uses two tokens, so both must stay balanced.

- Poster must keep equal bull and bear tokens
- Script warns if balance difference is more than 5%
- Rebalancing is done manually (auto option later)
- Even slightly balanced users still have good influence

6.5 Poster Configuration and Operational Hardening

Feature	Implementation
Price source	Uses CoinGecko, Binance as backup
Gas strategy	Uses EIP-1559 with limited max gas
Nonce handling	Tracks nonce per chain and resets if needed
Retry system	Retries 3 times, then skips temporarily
Logging	Stores logs with price, status, and gas
Alerts	Sends alerts on failure or low weight
Process	Runs with PM2 and Docker

6.6 Poster — Responsibilities:

Task	What it does
Fetch price	Gets latest price from APIs
Submit price	Sends price to Karma contract
Balance check	Checks bull and bear balance
Warning system	Alerts if balance is uneven
Multi-chain support	Works across all configured chains
Error handling	Handles failures and retries
Logging	Records all actions and results
Monitoring	Sends alerts for issues
Automation	Runs continuously without manual work

7. TESTING AND QUALITY STRATEGY

How we guarantee correctness — unit, fuzz, integration, and security

7.1 Test Coverage Plan

Test Type	Tool	Coverage	What is tested
WeightLib tests	Foundry	Full coverage	Zero, balanced, biased, edge cases
PriceAverager tests	Foundry	Full coverage	Single/multiple updates, decay logic
submitPrice tests	Foundry	All cases	Invalid input, events, state updates
Fuzz — WeightLib	Foundry	10k+ runs	Weight stays in valid range
Fuzz — Averager	Foundry	10k+ runs	No errors, decay works correctly
Flash loan test	Foundry fork	Manual	Flash loan has near-zero effect
Sybil test	Foundry script	Manual	Many small users don't affect price
Full integration	Foundry	End-to-end	Full flow and price accuracy
Security check	Slither	CI check	No major issues before deploy
Frontend tests	Vitest	All components	UI shows correct data
Poster tests	Jest	Core logic	Fetch, retry, balance checks

7.2 Gas Profiling

Function	Expected Gas	How it is optimized
submitPrice()	~45k gas	Uses library and keeps storage updates minimal
getPrice()	~2.5k gas	Read-only, no storage writes
latestRoundData()	~3k gas	Simple wrapper over getPrice
computeWeight()	~200 gas	Pure calculation, no storage
update()	~20k gas	Only needed storage updates are done

8. MULTI-CHAIN DEPLOYMENT STRATEGY

Deploying and verifying Karma on all 5 EVM chains

8.1 Target Networks

Chain	Explorer	Why this chain	Deployment plan
Ethereum Classic	blockscout	Main focus for project	Deploy first after testing
Ethereum	etherscan	Standard for testing	Use Sepolia before mainnet
Polygon	polygonscan	Low gas, fast transactions	Deploy after ETH
Base	basescan	Growing L2 ecosystem	Deploy after Polygon
BSC	bscscan	Large user base	Deploy with custom gas setup

8.2 Deployment Checklist (Per Chain)

Step	What to do	How
1	Deploy WeightLib	Run forge deploy script
2	Deploy PriceAverager	Run forge deploy script
3	Deploy Karma contract	Pass tokens, lambda, balance params
4	Verify contract	Verify on explorer
5	Save ABI	Auto saved after deploy
6	Update poster config	Add contract address in .env
7	Test contract	Submit price and check output
8	Update frontend	Add contract address in config

- Deployment is done step-by-step to ensure each part works correctly before moving ahead.
- After deployment, testing and configuration make sure the system runs smoothly across all chains.

9 TECHNOLOGY STACK

Every tool, every version, their reasons

Layer	Tech Used	Version	Reason
Smart Contracts	Solidity	0.8.x	Safe and reliable, handles errors well
Contract Framework	Foundry	Latest	Fast testing and already used in project
Contract Libraries	OpenZeppelin	v5.x	Trusted and secure standard code
Frontend	Vite + React	Latest	Already used, easy to extend
Wallets	RainbowKit + wagmi	v2	Supports many wallets and chains
Contract Calls	viem	v2	Clean and type-safe interaction
Off-chain Script	Node.js + TypeScript	Latest	Same setup as existing system
Price Source	CoinGecko + Binance	Free APIs	Easy to use and no cost
Contract Testing	Foundry	Latest	Covers unit, fuzz, and full tests
Frontend Testing	Vitest	Latest	Fast and simple testing
Poster Testing	Jest	Latest	Good for backend scripts
Security	Slither	Latest	Finds common vulnerabilities
CI/CD	GitHub Actions	—	Auto test and deploy on changes
Frontend Deploy	Vercel	—	Easy and fast deployment
Contract Deploy	Foundry Scripts	—	Simple and repeatable deploy
Docs	NatSpec + Markdown	—	Clear and structured docs

10 IMPLEMENTATION TIMELINE — 10 WEEKS

Phase Structure

Phase	Weeks	Focus	Completion Signal
Phase 1 :Contracts	W11–W15	WeightLib, PriceAverager, Karma.sol, full test suite, Sepolia deploy	All tests green; contracts verified on Sepolia; 80%+ coverage
Phase 2 : Multi-chain	W16–W17	Security audit, gas optimisation, deploy to all 5 chains	Contracts live + verified on ETC, ETH, Polygon, Base, BSC
Phase 3 :Frontend and Poster	W18–W23	Extend Fate frontend; build poster script; E2E integration test	Live dApp on Vercel; poster submitting on all 5 chains

Note: All weeks assume approximately 30–35 hours of focused work. Timelines may shift slightly based on mentor feedback and review.

Week	Phase	Tasks and Goals	Deliverables
Phase 1 — Smart Contract Development (Weeks 1–5)			
Week 11	Foundation and Interfaces	Community bonding and deep study of Fate-Solidity, OrbOracle-Solidity, and GluonReactor.sol. Set up a Foundry project with a GitHub repository and CI skeleton. Work closely with mentors to finalise the Karma.sol interface, then author IKarmaOracle.sol and IWeightLib.sol.	Foundry scaffold; IKarmaOracle.sol; GitHub Actions CI; signed-off interface contracts
Week 12	Core Logic	Extract and adapt GluonReactor.sol's weight function into WeightLib.sol implementing computeWeight(uint256 bull, uint256 bear). Simultaneously, adapt OrbOracle's decayed averaging into PriceAverager.sol with update() and getPrice().	WeightLib.sol; PriceAverager.sol; fuzz tests I1–I5 all green; 100% branch coverage;
Week 13	Karma.sol Build	Implement Karma.sol: inherit FatePool, implement IKarmaOracle, and wire WeightLib with PriceAverager. Build out submitPrice() with a MIN_BALANCE guard, nonReentrant modifier, and full event emission. Implement getPrice() and a latestRoundData() wrapper for Chainlink compatibility.	Karma.sol; integration test passing; end-to-end flow verified
Week 14	Security Hardening	Simulate a flash-loan attack on an Anvil fork and confirm weight approaches zero for flash borrows. Run a Sybil flood simulation with 100 wallets holding tiny balances. Add price range validations.	Security test suite passing; zero Slither high-severity findings; gas profile documented
Week 15	Testnet Deployment and Mid-Term	Deploy all contracts to Ethereum Sepolia, verify on Etherscan, and export ABI JSON. Write Deploy.s.sol scripts for both Hardhat and Foundry. Compile the mid-term evaluation package: architecture document, ABI exports, test-coverage report, and Slither report. Hold a mentor review session.	Contracts deployed and verified on Sepolia; mid-term evaluation submitted; ≥80% test coverage

Week	Phase	Tasks and Goals	Deliverables
Phase 2 — Multi-Chain Deployment (Weeks 6–7)			
Week 16	Testnet Roll-out	Deploy Karma to Ethereum Classic (Mordor testnet), Polygon Mumbai, and Base Sepolia. Verify contracts on Blockscout, Polygonscan, and Basescan. Update the frontend chain config and run the poster for each chain.	Karma live on ETC, Polygon, and Base testnets; chain-quirks documentation
Week 17	Mainnet Deployment	Deploy to BSC testnet and complete a 5-chain smoke test and running the poster across all chains and confirming every submitPrice() call succeeds. Begin mainnet deployments starting with ETC mainnet (priority chain).	Contracts live and verified on all 5 chains; full deployment manifest in /deployments/
Phase 3 — Frontend and Poster (Weeks 8–10)			
Week 18	Hooks and Components	Study the Fate-EVM-Frontend component tree. Extend the chain config with all 5 Karma addresses. Build a useKarma.ts wagmi hook covering getPrice, the submissions mapping, and token balances.	useKarma.ts hook; PriceSubmit.tsx rendering with weight preview
Week 19	Dashboard and Leaderboard	Build OracleDash.tsx (current price, staleness indicator, submitter count) and MyPosition.tsx (balances, weight, recommendation). Wire all components to all 5 chains via wagmi's chain-switching.	OracleDash; MyPosition; multi-chain working; Leaderboard live with real event data;
Week 20	Poster and E2E Integration	Build the Karma Poster from the OrbOracle-Poster base. Implement the 5-chain loop with CoinGecko/Binance price fallback, nonce management, retry logic, and structured JSON logging. Write Jest tests for core poster logic.	Poster operational on all 5 mainnets; PM2/Docker config; Jest tests passing; second mid-term submitted
Phase 4 :Polish, Security and maintaining application onwards (Weeks 11–12)			
Week 21	Performance and Security	Optimise performance: add GraphQL query caching, lazy-load the leaderboard, and batch balance reads with multicall. Gas-profile remaining on-chain inefficiencies from real mainnet submissions and document findings. Run a manual security audit of Karma.sol, WeightLib, and PriceAverager.	Performance report; gas optimisation report; security audit notes; patched issues; hardened contracts
Week 22-Onwards	Docs, Demo and Final Submission	Deploy the frontend to Vercel with production environment variables. Harden the GitHub Actions pipeline (lint + build + test + Slither + deploy on merge to main)Write the final GSoC report, incorporate mentor feedback, code freeze, and submit the final evaluation. Prepare handoff docs: contributing guide, known issues, post-GSoC roadmap.	Complete NatSpec docs; production Vercel deployment; CI/CD pipeline; integration guide; architecture diagram;

13 POST-GSOC ROADMAP

What Karma becomes after the summer — the long-term vision

Feature	What it means	Why it matters
Karma as shared oracle	Other apps use Karma for price	One system serves whole ecosystem
Governance for decay	Users vote to change decay rate	Adjust speed vs stability
Gasless submission	Users sign, relayer pays gas	Easy for small users to join
Auto rebalancing	Poster auto keeps 50/50 balance	No manual work needed
Cross-chain oracle	Combine prices from all chains	More secure and reliable
Beyond price data	Use for events or other data	Works for more use cases
ZK weight proofs	Hide balances using proofs	More privacy for users

14 PAST EXPERIENCE and OPEN SOURCE CONTRIBUTIONS

The following project contributions tells direct technical alignment with DIT's requirements: Solidity contract development, EVM tooling, React/TypeScript frontends, and open-source project workflows. All contributions listed below in next table.

Organization	Language / Stack	Contribution Type	Relevance to DIT and Karma
Saving-Satoshi	TypeScript / JavaScript	Bug fixes, UI component enhancements to Bitcoin education platform	React frontend patterns; open-source PR workflow
Bitcoin-Dev-Project	TypeScript / Markdown	Documentation, tooling improvements for Bitcoin developer education	Technical writing; developer tooling; Web3 ecosystem
AOSSIE	Next.js/Python/Devops	Web3 frontend components	dApp frontend, Frontend and Devops
Stability Nexus	Solidity / DeFi	Solidity contract development and Web3 integrations in DeFi projects	Direct: Solidity 0.8.x, EVM patterns, DeFi contracts

Contributions to the Aossie + StabilityNexus + DjedAlliance

(Decemeber 2025-March 2026) – 4 months

114

Total PRs

77

MERGED

23

Open

14

Closed

10.1 Pull Requests: ALL MERGED PRS (Listed: Complexity High to Low) (AOSSIE-Org + StabilityNexus + DjedAlliance):

SN.no.	Status	Title	Description	Link
01.	 MERGED	Deterministic Preprocessing Verification Mode — cryptographic pipeline re-validation	Addresses Gap 5.1 from framework paper. CLI re-runs full preprocessing from scratch, validates all SHA256 hashes and Merkle roots against manifest. Auto-detects bz2/uncompressed dumps.	Link
02.	 MERGED	Add Merkle Proof Generation for Chunk-Level Dataset Verification	Implements generate_merkle_proof() and verify_merkle_proof() enabling cryptographic proof that a specific chunk belongs to the dataset without re-hashing the full file. Merkle root and chunk size added to manifest.	Link
03.	 MERGED	feat: complete MiniChain core — PoW, Ed25519, smart contract engine	Full blockchain node: Proof-of-Work mining, Ed25519 transaction signing, account state, Python smart contract sandbox with deploy/call lifecycle, mempool, P2P stub. 5 unit tests all green.	Link
04.	 MERGED	Core foundation for the Identity Tokens protocol: MVP Implementation	Introduces IdentityToken.sol (ERC-721 self-minting, attribute storage, endorsements), Deploy.s.sol script, and IdentityToken.t.sol test suite. mint(), setAttribute(), endorse() all tested via make test.	Link
05.	 MERGED	Implement Manifest Chain Tamper Detection for Cryptographically Linked Dataset Manifests	Each new manifest stores the SHA256 hash of the previous manifest in parent_manifest_hash field. If any earlier manifest is modified, the stored hash no longer matches and verification fails. CLI accepts optional --previous-manifest for explicit chain verification. Comprehensive test suite covering chain creation, tamper detection, and backward compatibility.	Link
06.	 MERGED	Implement Various Identity Query Functions	Added five read functions to the IdentityToken contract: hasIdentity() checks if an address holds any DIT token, getIdentity() returns full metadata struct, getIdentityByOwner() returns all token IDs for an address, isVerified() checks for at least one active endorsement, and isExpired() checks if validUntil is past block.timestamp. Closes #52.	Link
07.	 MERGED	Introduce centralized file-based logging infrastructure using Winston	Added src/utils/logger.ts with timestamped JSON structured logs to ~/.rein/log.txt. Integrated across WebSocket server. Features: exception/rejection handling, dev console logging, duplicate message safeguards.	Link

08.	 MERGED	Availability: JSON DoS Vulnerability — oversized payload attack vector	Added data.length check before JSON.parse(raw) to reject messages larger than 10KB. Prevents attacker from sending 100MB JSON that blocks the single-threaded server, causing Denial of Service for all users.	Link
09.	 MERGED	Token Owner Should Be Able to Remove a Metadata Field Entirely	Implements deleteAttribute() which checks ownership via onlyTokenOwner, blocks compromised identities via notCompromised, hashes the string key the same way setAttribute stored it, wipes the slot entirely, and emits AttributeDeleted event for off-chain indexers. Includes tests for deletion, event emission, repeated deletions, access control, and compromised identity blocking. Closes #39.	Link
10.	 MERGED	Integrate RainbowKit and wagmi to Enable Secure, Multi-Wallet Connection	Added multi-wallet connection via RainbowKit and wagmi. Dynamic wallet button (connect/wrong-network/account states). App-wide provider with dark theme. Fixed package-lock.json dependency mismatch build errors.	Link
11.	 MERGED	Buggy client or attacker sends NaN as coordinate — crashes app in evaluations	Added Number.isFinite(msg.dx) and Number.isFinite(msg.dy) guards before all math operations. Prevents NaN coordinates from reaching nut.js OS automation library which throws unhandled Node.js process exceptions.	Link
12.	 MERGED	Incremental Preprocessing with Checkpoints to Avoid Halfway Fail-Restart	Saves progress checkpoints during Wikipedia dump preprocessing so a failed run resumes from where it left off rather than restarting from scratch. Checkpoint file auto-deleted on successful completion. Manifest hashing excludes the predecessor reference so hashes reflect content only. Fixes #63.	Link
13.	 MERGED	Major Interface Reengineering: Responsive Controls, Dynamic Keyboard Toggle, Layout Optimization	Breaking change. Expanded keyboard to 6x6 grid (36 keys) with Play/Pause toggle. New icon set. Compact 6-button control bar grid. Visual status indicators: connection strip, tracking spinner, SCROLL badge.	Link
14.	 MERGED	Decentralized EVM Identity Framework with Governance, Trust Graphs, and Social Recovery	Introduced ERC-721 identity token system with Foundry tooling, modular attribute registry, trust graph endorsements, 14-day timelocked social recovery, and full CI/CD automation. All forge tests pass.	Link
15.	 MERGED	Architectural Improvement of Settings Page for Adaptive Multi-Section Desktop Rendering	Two-column responsive settings layout: controls left, sticky QR code + trackpad link panel right. Improved container sizing, heading spacing, visual hierarchy. Warning alert and footer repositioned.	Link
16.	 MERGED	Add Merkle Tree Based Chunk-Level Hashing for Dataset Verification	compute_merkle_root() splits files into 1MB chunks, hashes each with SHA256, builds tree bottom-up, stores raw_merkle_root and processed_merkle_root in dataset manifest. 16 tests pass in 1.88s.	Link
17.	 MERGED	Major Architectural Refactor: Modularization for Long-Term Scalability	Complete Next.js App Router restructure with 4 feature pages. Custom hooks: useScrollHiddenNav, useTypewriter. Centralised lib/constants.ts and lib/fonts.ts. Animated navbar, typewriter hero, animated theme toggle.	Link
18.	 MERGED	Introduce end-to-end automated governance layer — CI, static analysis, quality enforcement	Added CI workflow running formatting, build, tests, and Slither static analysis on every push and PR. Added Prettier ignore file, pre-commit hooks, and npm scripts for Solidity formatting/linting.	Link

19.	 MERGED	Refactor Root Layout by Modularizing Fonts, Metadata, and Global Providers	Added 3 custom web fonts (Atyp, Garamond, UtSaHa) from WOFF2 files. Centralised font config in lib/fonts.ts. Simplified root layout to use shared font object. Eliminates duplicated font setup. Closes #40.	Link
20.	 MERGED	Extend Merkle proof system — portable proof export and standalone verification	Adds export_merkle_proof(), load_merkle_proof(), verify_merkle_proof_from_file() so proofs can be saved as JSON, shared, and independently verified. Rejects malformed, tampered, or invalid proofs. Full negative test coverage	Link
21.	 MERGED	Automated Issue Duplication Detection and Pull Request Ownership Validation	Added GitHub Actions workflow that analyzes new/edited issues for similarity, posts analysis comments, and labels highly similar issues (80% threshold) as duplicates. Second workflow validates that PRs are properly linked to issues and that authors are assigned to the linked issues. Fixes #216 and #217.	Link
22.	 MERGED	Create IdentityToken.sol Interface --- IIdentityToken.sol	Added IIdentityToken.sol interface file declaring all public and external functions of IdentityToken including minting, attribute management, endorsement tracking, and identity state management. Includes getters for public state variables and standard ERC721 view functions. Allows other contracts to interact with IdentityToken without importing the full implementation. Closes #21.	Link
23.	 MERGED	Implement Dynamic Mouse Sensitivity Scaling and Configuration Persistence	Added sensitivity multiplier (0.1x-3.0x) in InputHandler.ts. Settings page gets Slow/Default/Fast slider. Preference persisted in server-config.json across restarts. Fully verified at min, max, and default values.	Link
24.	 MERGED	Update Chain Configuration to Automatically Detect User Active Wallet Network	Added Citrea Testnet configuration to the active chains list with RPC endpoint and dedicated icon. Application now auto-selects the network active in the user wallet. Re-enabled BatchPayment module to restore batch payment processing. Closes #53.	Link
25.	 MERGED	Harden InputHandler Against Event Flooding and Oversized Payload Attacks	Fixed InputHandler.ts: rate limiting (~60 FPS throttle), text length cap (500 chars), coordinate clamping (+-2000px), invalid key/combo guards, improved error logging. Prevents DoS via event flooding.	Link
26.	 MERGED	Smart Contract Optimization via Custom Errors and Structural Refactoring	Refactored Chainvoice.sol to replace expensive string revert messages with gas-efficient custom errors across all functions. Optimized contract modifiers and internal naming to resolve linting warnings.	Link
27.	 MERGED	Test Suite Synchronization --- HammerAuctionHouse-Solidity	Fixed three mismatched revert messages in tests to match actual contract modifier strings. Added missing test case for zero auctioned token address. Updated error strings: empty name to String must not be empty, zero address to Address must not be zero, double-withdrawal to Auctioned asset has already been claimed. All tests pass.	Link
28.	 MERGED	Use Promise.all in English-Auction to Fetch Token Names in Parallel	Replaced sequential await getTokenName() calls with Promise.all() in mapAuctionData of english-auction-service.ts so both auctionedTokenName and biddingTokenName are fetched simultaneously.	Link
29.	 MERGED	Modular Core Implementation and Smart Contracts --- MiniChain	Full blockchain node implementation: Proof-of-Work mining, Ed25519 transaction signing, account-based state management, Python smart contract sandbox with deploy/call lifecycle, mempool, and P2P stub.	Link

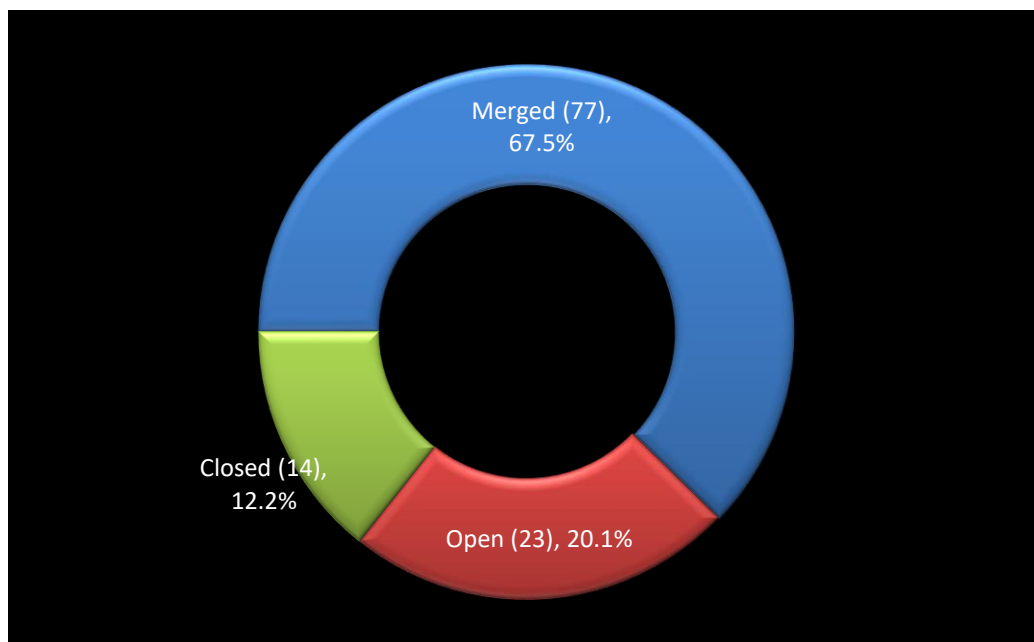
30.	 MERGED	Added QR Code for Android App Download	Desktop users seeing the Google Play button can now scan a dynamic QR code placed beside it to open the Play Store link directly on their phone. Adds visual glow effect for QR presentation. Improves mobile-from-desktop acquisition flow.	Link
31.	 MERGED	Enhanced Control Experience with Dynamic Keyboard Accessibility and Overall Refactor	Added Copy/Paste buttons to control bar. Collapsible extra-keys panel as 6x6 grid with media, navigation, modifier, and function keys. Keyboard toggles open/closed; extra-keys syncs with keyboard state. Control bar redesigned with unified icon-based buttons and simplified styling.	Link
32.	 MERGED	Adding events to src/ProtocolParameters.sol	Added FeeUpdated(uint256 newFee) and TreasuryUpdated(address indexed newTreasury) events emitted by updateFee() and updateTreasury(). Enables on-chain observability and event indexing for protocol changes.	Link
33.	 MERGED	Prettier with solidity plugin — replace forge fmt with Prettier	Replaced forge fmt with Prettier + prettier-plugin-solidity in CI. Added npm scripts sol-fmt-all and sol-fmt. CI now fails fast on formatting issues. Predictable, configurable formatting with zero logic changes.	Link
34.	 MERGED	feat: bootstrap Foundry contracts with CI workflow and configuration	Initialized Foundry project with foundry.toml, Counter.sol, Counter.s.sol deployment, Counter.t.sol fuzz tests, and GitHub Actions CI. Build, test, and validation automated. Closes #6.	Link
35.	 MERGED	fix multiple Biome lint errors, a11y warnings, and formatting issues	Replaced all noExplicitAny with unknown across 6 files. Added SVG aria-label/title accessibility. Fixed duplicate JSX props, useless else, node: protocol imports, non-null assertions. Removed tracked .vscode folder.	Link
36.	 MERGED	Passing an argument of a different type triggers unhandled runtime exception	Added type guards for route argument validation in home and chat routes. Prevents crashes from unexpected argument types passed to route handlers. Fixes #177.	Link
37.	 MERGED	added-meta-keyword-for-search-engines	Added meta keywords tag to help LLM-based crawlers (ChatGPT, Gemini) discover the Resonate website. Keywords visible in page source to all web crawlers. Closes #116.	Link
38.	 MERGED	Performance Optimization: useMemo in TransactionsPage to memoize statistic calculations	Added useMemo for transactionStats in TransactionsPage. Prevents expensive recalculations on every re-render (modal open). Passes [transactions] as dependency so stats auto-update on data change.	Link
39.	 MERGED	Infinite Swap Execution Loader With Missing Failure Handling	Wrapped contractClient.buy/sell/swap in try-catch. Missing error handling caused loading state to stay active forever on user rejection. Loading state and preview visibility now properly reset on error.	link
40.	 MERGED	CTASection Footer Links Unresponsive Due to Overlay Capturing Pointer Events	Fixed CSS overlay z-index issue blocking footer links. Added animated hide-on-scroll navbar, dark mode theme toggle with smooth animations, typewriter hero animation, and centralised component barrel exports.	Link
41.	 MERGED	Integrate Prettier, ESLint, and Husky pre-commit — Add 404 Page	Added 404 Not Found page for successful Next.js build. Added Prettier with Tailwind plugin, ESLint-Prettier integration, Husky pre-commit hooks, lint-staged, and formatting scripts.	Link

42.	 MERGED	Fix pool instantiation caused by undefined contract reads or NULL read	Improved pool instantiation detection and error handling. Returns explicit not-instantiated results when contract data is missing or invalid. Prevents silent failures and incorrect status.	Link
43.	 MERGED	Fixes issue 16 — README and contributing guidelines	Restructured README with identity protocol descriptions, tech stack (Foundry, OpenZeppelin, Wagmi), installation commands, build/test procedures. Comprehensive contributing guidelines with branching strategy and PR criteria.	Link
44.	 MERGED	Integrate Vercel Preview Deployments in CI Workflow	Enabled Vercel PR previews via Vercel GitHub integration. Removed GitHub Pages previews to avoid duplication. Configurable base path support. Auto-cleanup on PR close.	Link
45.	 MERGED	CI: Slither-fix, Fixes issue 19	Bumped Solidity compiler to 0.8.23 project-wide across contracts, scripts, and tests. Consistent pragma updates across all artifacts. No functional changes — compiler version alignment and Slither static analysis fix.	Link
46.	 MERGED	Browsers will throw a QuotaExceededError if cache is full — localStorage crash fix	Wrapped localStorage.setItem in try-catch in saveTransaction method. Added user-facing message when storage exceeds quota and generic error notifications for other save failures. Prevents frontend script halt.	Link
47.	 MERGED	Update Configuration file for automated code quality enforcement	enabling comprehensive automated code review, static analysis, and quality checks as recommended in the AOSSIE Discord channel.	Link
48.	 MERGED	fixing-bug — Fixes #40	Fixed bug reported in issue #40 in the TNT project. Targeted code fix verified with no new warnings or errors. All existing tests and linting checks pass.	Link
49.	 MERGED	client-server-settings — Fixes #55	Migrated mouse sensitivity and scroll inversion from server-config.json to browser localStorage per-device. Server receives pre-calculated deltas.	Link
50.	 MERGED	added-dependent-bot-for-dependency-update --- Resonate-Website	Integrated Dependabot for automated dependency management on the Resonate-Website project. Bot periodically checks for outdated or vulnerable dependencies and raises update PRs. Scheduled dependency checks enabled with no runtime changes to CI. Closes #188.	Link
51.	 MERGED	server is of type unknown in createWsServer — TypeScript type safety fix	Fixed TypeScript type error in createWsServer: server typed as any replaced with proper annotation. Enforces compatibility validation at compile time. Prevents undetected runtime type mismatches in WebSocket server.	Link
52.	 MERGED	Update ExtraKeys.tsx to replace Backspace with Enter key	Replaced Backspace button with Enter key in ExtraKeys.tsx (Closes #210). All functional verifications passed: authentication, gestures, cursor mode, scroll mode, sensitivity, key combinations, glide typing, voice input.	Link
53.	 MERGED	Remove Redundant Lock Files and Standardize Package Management — normal ConnectButton	Removed redundant lock files. Replaced custom wallet connection component with default RainbowKit ConnectButton, reducing implementation complexity while maintaining full functionality. Closes #51.	Link

54.	 MERGED	Fixes issue 30 — token approval flow confirmation	Token approval flows now wait for on-chain transaction confirmation before reporting success. Fixes premature completion bug. Improves reliability and consistency for all token approval interactions.	Link
55.	 MERGED	Update README with tech stack and installation steps	Added detailed Tech Stack section (Next.js, Tailwind, shadcn/ui, wagmi, React Hook Form). Getting Started with clone/install/navigate commands. Usage with dev/build commands and Contributing workflow.	Link
56.	 MERGED	CI/CD Workflow: non-mandatory fields made optional as per issue 29	Updated GitHub Issue Forms for Bug Report and Feature Request templates. Marked all optional fields as required:false, labeled optional sections, removed unnecessary placeholders for cleaner issue previews.	Link
57.	 MERGED	CI/CD setup for project to check PR build and linting	Added GitHub Actions workflow validating code quality on PRs via ESLint linting and build checks. Prevents broken builds from being merged. Closes #22.	Link
58.	 MERGED	Enhancement: Add Comprehensive Functional Validation and Verification Checklist	Updated PR template with full functional verification checklist: basic gestures, cursor/scroll modes, sensitivity, key combos, keyboard input, glide typing, voice input. Documentation-only change.	Link
59.	 MERGED	Deleted the older coderabbit.yml file to prevent duplicate configuration conflicts	Removed legacy .coderabbit.yml conflicting with the newer config. Duplicate configuration was causing redundant review triggers and inconsistent automated review behaviour. Clean-up only.	Link
60.	 MERGED	Include Complete Installation and Development Workflow Guidelines	Added CONTRIBUTING.md with setup, code standards, and PR requirements. Added hide-on-scroll navbar, animated theme toggle, typewriter hero	Link
61.	 MERGED	Fix formatting issues causing build failures	Fixed CI/CD by reformatting all files with Prettier, redefining build commands, and updating next.yml. Previous misconfiguration caused every PR build to fail.	Link
62.	 MERGED	Removing deploy command on each PR — build is sufficient on pull requests	Removed GitHub Pages deployment from pull_request workflows; limited to push on main. OIDC unavailable on PRs makes deploy unnecessary. Disabled Dependabot scheduled PRs.	Link
63.	 MERGED	Dependabot setup — automated dependency updates	Configured Dependabot to scan npm and GitHub Actions for newer dependency versions and auto-create update PRs. Added CI workflow for Node.js build validation. Closes #11.	Link
64.	 MERGED	Enhance CONTRIBUTING.md PR Rules	Restricted direct commits to main for internal team only. All external contributions must go through reviewed pull requests. Added validation, auditing, and unauthorised-change prevention guidelines.	Link
65.	 MERGED	fix double charge in _postSell	Removed duplicate processProtocolFees() call in _postSell of Maelstrom.sol that was double-charging protocol fees on every sell transaction. Critical financial correctness fix.	Link

66.	 MERGED	Fix: An empty pool (0 ETH) leads to division by zero errors	Added require(msg.value > 0), require(amountToken > 0), require(initialPriceBuy > 0 and initialPriceSell > 0) guards to initializePool(). Prevents silent division-by-zero in AMM pricing logic.	Link
67.	 MERGED	Fix Dependabot YAML unsupported key	The security-updates option exists in GitHub repository settings, not in dependabot.yml.	Link
68.	 MERGED	Add GNU General Public License v3 --- Chaininvoice	Added the GNU General Public License v3 (GPLv3) to the Chaininvoice project, establishing the open-source licensing terms for the codebase. Closes #17.	Link
69.	 MERGED	Fix Typo in Error Message for Current Bid	Fixed typo in error message displayed during auction operations in HammerAuctionHouse-WebUI, improving clarity for users encountering bid validation errors.	Link
70.	 MERGED	added basic CI/CD workflow with npm ci and npm run build steps	Added basic CI/CD workflow with npm ci and npm run build steps. Configured automated pipeline for Next.js site deployment to GitHub Pages, triggered on commits to main with build caching optimizations. Closes #10	Link
71-77 (8 PRs)	 MERGED	---- Similar PRs which aims to put Coderabbit configuration files in different repos ----	ALL COLLECTED LINKS ARE HERE	LINKs
•	 MERGED	These are some recent contributions done which are not listed above	ALL COLLECTED LINKS ARE HERE	LINKs

PR Status Distribution (Dec 2025 – Mar 2026)



Open Pull Requests — Awaiting Review / Merge-Ready

StabilityNexus · DjedAlliance

23

Currently Open PRs

19

Active / In Review

4

Draft / WIP

SN.no.	Status	Title	Description	Link
01.	○ OPEN	MVP WindmillExchange — Modular Structure with Libraries — Windmill-EVM-Contracts	Full modular on-chain DEX: WindmillExchange.sol with createOrder, cancelOrder, matchOrders. Separate libraries: MathUtils (512-bit mulDiv), PriceCurve (linear price-over-time), TokenTransfer, OrderStorage, PairStorage. CEI pattern throughout, inline reentrancy guard, midpoint settlement, 15 custom errors. Full test suite with fuzz tests.	Link
02.	○ OPEN	Generalize StablePay: Multi-Stablecoin + ERC20 Base Assets Support — StablePay	Added Djed Isis/Tefnut variant support. ERC20 base asset buy/sell flows (zero msg.value). Oracle support for Orb, Chainlink, HebeSwap, API3. Atomic exits via sellBothCoins. Refactored networksConfig for arrays of stablecoins per network. TokenDropdown and TransactionReview updated for ERC20 approval flow. Closes #84.	Link
03.	○ OPEN	Remove Dead Milkomeda RPC Logic — StablePay	Removed Milkomeda Mainnet blockchain support including network configuration, chain definitions, and error handling mappings. Eliminates dead code and reduces maintenance surface area. Closes #74.	Link
04.	○ OPEN	MVP — WindmillExchange Single-File Architecture — Windmill-EVM-Contracts	Clean 3-file MVP: WindmillExchange.sol (types, math, price curve, token transfer, exchange logic), test/WindmillExchange_t.sol (unit tests, edge-case reverts, fuzz), script/DeployWindmill_s.sol (Forge deploy script). All tests pass. Closes #1.	Link
05.	○ OPEN	Enhance Protocol Observability via Event Updates — Djed-Solidity	Added postRatio, currentPrice, and newReserveBalance to all core trading events (BoughtStableCoins, SoldStableCoins, BoughtReserveCoins, SoldReserveCoins, SoldBothCoins) in Djed.sol and DjedShu.sol. Enables real-time monitoring without extra blockchain queries. Closes #50.	Link
06.	○ OPEN	Enable Multicall Batching — StablePay-MerchantDashboard	Added batch: { multicall: true } to wagmi-config.ts. Enables batching of multiple contract reads into a single RPC call, optimizing network requests and improving dashboard	Link

SN.no.	Status	Title	Description	Link
			performance. Closes #50.	
07.	○ OPEN	Dynamic Wallet-State Binding — StablePay-MerchantDashboard	Added walletAddress to useEffect dependency array so it re-runs when wallet changes. Per-wallet cached transaction history, isolated state, and cache expiry. Prevents cross-wallet state leakage. Closes #40.	Link
08.	○ OPEN	Refactor djed-sdk tradeUtils.js — StablePay	Fixed transaction sending to consistently use Web3.js instead of Ethers.js syntax. Improved verification.	Link
09.	○ OPEN	Optimize Token Metadata Fetching — Caching + Multicall — Maelstrom-WebUI	Added in-memory caching so already-fetched token data is reused. Combined multiple RPC calls (name, symbol, decimals) into one efficient multicall. Added Wagmi client safety check. Faster page loading, fewer requests, prevents HTTP 429 errors. Closes #36.	Link
10.	○ OPEN	Fix Stale State in Output Calculation — Maelstrom-WebUI	setSwapState called calculateOutput while the new price was queued in state but not yet committed, using the previous token's price. Fix: pass freshly-fetched price directly as an argument to calculateOutput instead of reading from stale state. Closes #55.	Link
11.	□ DRAFT	[DRAFT] Pagination for getSentInvoices / getReceivedInvoices — Chaininvoice	[DRAFT] Unbounded loops on large invoice arrays caused potential DoS and gas exhaustion. Added pagination parameters to invoice queries for efficient retrieval of specific subsets. Frontend updated to use paginated fetching. Test coverage updated. Closes #120.	Link
12.	□ DRAFT	[DRAFT] Contract Integrity CI: Gas, ABI, Security Scans — Chaininvoice	[DRAFT] Full CI suite: gas snapshot diff, gas report, contract size check, coverage, Mythril security scan, 4naly3er, storage layout diff, ABI diff check, deployment dry-run. Baseline gas snapshot added for regression detection. Closes #145.	Link
13.	○ OPEN	Fix Timezone-Unsafe Date Formatting in Financial Docs — Chaininvoice	Replaced raw new Date(invoice.issueDate) with UTC-aware formatting using date-fns/dayjs. An invoice issued on "Feb 20" in India no longer shows as "Feb 19" in New York. Missing or unparseable dates display "N/A" / "Invalid date". Table dates show localized date+time with UTC offset. Closes #121.	Link
14.	○ OPEN	Fix Direct DOM Manipulation — Use React State for Fallbacks — Chaininvoice	Replaced e.target.style.display="none" with React state (useState) for image fallbacks across InvoicePreview.jsx, CountryPicker.jsx, ReceivedInvoice.jsx. Broken logos now show a consistent gradient placeholder. Bypassing React's virtual DOM caused rendering inconsistencies. Closes #122.	Link
15.	○ OPEN	Fix ETH Balance Check Ignoring Gas Costs — Chaininvoice	Logic checked ethBalance < BigInt(fee) but ignored gas costs. A user with exactly the fee amount would fail the balance check because Fee + Gas is required. Fix adds gas buffer to the balance requirement and updates error messages to display the total required amount including gas. Fixes #123.	Link
16.	□ DRAFT	[DRAFT] Implement Maximum Minting Cap or Multi-Sig for Bulk Mints — TNT	[DRAFT] Configurable mint cap for tokens with deploy UI field and on-chain enforcement on issuance. UI shows mint progress and disables inputs when cap is reached. Clearer error handling and toasts when token creation or local save fail. Closes #35.	Link
17.	○ OPEN	Fix Infinite Loading State — Irrecoverable UI Deadlock —	Fixed infinite loading state in swap flow caused by missing error handling. Loading state and any irrecoverable UI deadlock now	Link

SN.no.	Status	Title	Description	Link
		Maelstrom-WebUI	correctly resets when an error occurs. Closes #50.	
18.	○ OPEN	Extend Event Coverage Across Contracts — HammerAuctionHouse-Solidity	Added TreasuryUpdated event to ProtocolParameters.sol for immutable audit trail of sensitive admin changes. Added FeePaid event to EnglishAuction.sol to record exact protocol fee deducted during withdrawal. Closes #29.	Link
19.	○ OPEN	Fix ETH Accounting Mismatch in processProtocolFees — Maelstrom-Solidity	Corrected protocol fee accounting to ensure internal balance deductions are properly synchronized with treasury transfers. Prevents accounting mismatch that caused inconsistent fee processing state. Closes #10.	Link
20.	○ OPEN	Structured Development Flow: Contributing Guidelines — Maelstrom-Solidity	Added CONTRIBUTING.md covering setup instructions, development prerequisites, code formatting standards (Prettier + Solidity), testing procedures, and PR submission guidelines. Closes #26.	Link
21.	○ OPEN	Modular Oracle Architecture & Multi-Provider Support — Gluon-EVM	Decoupled StableCoinReactor from Pyth via unified IGluonOracle.sol adapter interface. Added GluonPythAdapter, GluonChainlinkAdapter, GluonOrbAdapter. All adapters enforce maxAge staleness checks. Updated factory to accept generic oracle adapter address. Unit and integration tests with mocked oracles. Closes #8.	Link
22.	□ DRAFT	[DRAFT] Gas Fee Optimization — SLOAD Reduction in payInvoice — Chaininvoice	[DRAFT] Previous payInvoice accessed storage fields 14 times (~14 SLOADs = ~1,400 gas). Fix copies invoice struct to memory once, then all 14 checks use MLOAD (3 gas each). Estimated savings: ~1,350+ gas per transaction. Closes #74.	Link

Contributions to Other organisations:

Saving-Satoshi: 10 Merged 1 Open

Bitcoin-Dev-Project: 2 Merged 0 Open

RUXAILAB: 1 Merged

Availability commitments:

1. **Weekly Pull Requests:** I commit to submitting at least one meaningful PR per week throughout the GSoC period.
2. **Mentor Meetings:** I will attend every scheduled mentor meeting, arrive prepared with updates, and follow up on all action items promptly.
3. **Proactive Communication:** I will keep mentors and the community informed of my progress, blockers, and plans no silent disappearances.
4. **Quality Standards:** Every PR I submit will include tests, pass all linters, and follow the project's code style and contribution guidelines.
5. **Documentation:** I will document my work clearly inline comments, PR descriptions, and updates to relevant docs — so the community can benefit long-term.
6. **Code Review Participation:** I will actively review other contributors' PRs, provide constructive feedback, and help maintain project quality.
7. **Milestone Accountability:** I will track my progress against the project timeline and flag risks early rather than missing deadlines silently.
8. **Post-GSoC Continuity:** I intend to remain an active contributor after GSoC ends maintaining code, reviewing PRs, and onboarding new contributors. **Would like to become maintainer for the upcoming gsoc sessions.**

*** Planned Low-Activity Period**

May : Weeks 2 and 3 (University Final Examinations)

My university final exams fall during this window. Contribution speed will be reduced, but I will **not disappear entirely**. Specifically:

- Will attend all mentor meetings and stay reachable on Discord / communication channels.
- Will review PRs, respond to comments, and keep open issues updated.
- New code contributions may slow down — I will compensate with extra effort in the weeks before and after.

Aossie Google-form contents

PERSONAL INFORMATION

Full Name	Aniket Kumar	Email	iamaniketkumarmaner@gmail.com
Phone	+91 6203026579	Country	India
GitHub	github.com/aniket866	Discord	aniket866
LinkedIn	linkedin.com/in/aniket-kumar-714551303/	Twitter	—

ELIGIBILITY

Age	18+ years old	AOSSIE Relative?	No
Prev. AOSSIE GSoC?	No	Prev. GSoC (other org)?	No
Conflicts of Interest	None		

EDUCATION & EXPERIENCE

Highest Degree	Completed High School	Currently a Student?	Yes
University	Panjab University, Chandigarh	Field of Study	Information Technology
Expected Graduation	May 31, 2027	SGPA (Last Sem)	9.2 / 10
Relevant Courses	• Software Project Management (ongoing) • Network Security & Cryptography (completed) • Machine Learning (ongoing) • Deep Learning (ongoing) • Internet & Web Programming (ongoing)		

CONTRIBUTIONS TO AOSSIE

Merged PRs	77	Pending PRs	23
Projects Contributed	DIT ,Windmill , Chainvoice , HammerAuction , StablePay , Open Verifiable LLM ,Rein , Resonate Website , Pictory , Minichain..... And many others		
Theme Contributed	Decentralized Economic and Financial Stability		
Best PR #1	github.com/AOSSIE-Org/OpenVerifiableLLM/pull/25		
Best PR #2	github.com/StabilityNexus/Windmill-EVM-Contracts/pull/2		
Best PR #3	github.com/StabilityNexus/IdentityTokens-EVM-Contracts/pull/30		
AI Tools Used	CodeRabbit, ChatGPT	AI Usage Summary	Codebase understanding, edge-case optimisation, debugging

Aossie Google-form contents

GSOC WORK — OVERVIEW

Proposal Title	Order-\Book Auctin-Based Exchange and Karma Protocol		
Project Size	Large	Project Length	22 Weeks (Preferred by AOSSIE)
Summary	In this project, I will build a new type of trading system where orders are not fixed prices but change with time. Users can buy and sell tokens, and trades happen automatically when prices meet. Everything runs fully on blockchain, no central control, and anyone can help execute trades.. Along with this I will develop the Karma Protocolwhich is a trustless system to discover prices without relying on external oracles. It will use user participation and incentives to calculate fair prices based on market behavior.		
Detailed Description	Already discussed In proposal above		
Motivation	Already detailed in the proposal document		
Personal Motivation & Suitability	Already detailed in the proposal document		
Tech Stack	Already detailed in the proposal document		

TIMELINE (CODING WEEKS 1–8 DETAILED IN PROPOSAL DOC)

Bonding Period	Detailed in proposal doc	Coding Week 1	Detailed in proposal doc
Coding Week 2	Detailed in proposal doc	Coding Week 3	Detailed in proposal doc
Coding Week 4	Detailed in proposal doc	Coding Week 5	Detailed in proposal doc
Coding Week 8	Detailed in proposal doc	Weeks 9 – 22	Outlined in proposal doc

COMMITMENTS & ADDITIONAL INFO

AOSSIE Requirements	Will submit at least 1 PR/week (unless exception agreed with mentors) Will attend AOSSIE weekly meetings Will write blog posts and summaries		
AI Usage Disclosure	No AI used to write the proposal content. 2–3 UI mockup images generated with Nano-banana to illustrate the interface design.		
Community Sustainability	Attract contributors via beginner-friendly issues, demos, and junior mentoring. Maintain projects through regular updates, testing, and community involvement.		
Future Mentor in 2027?	Yes	How found AOSSIE	Previous year GSoC accepted organisations list
Why AOSSIE?	First of all, being honest, open source contributions help me learn much more in a practical way. AOSSIE stood out to me because it has a strong background in blockchain, which directly matches my interests. I want to build new and impactful applications in blockchain, and this organization provides the right environment for that. It is a good fit for me in both learning and contributing, as I can improve my skills while also creating something useful for the open source community and society. } This is not to be seen as personal intrest as I have contributed a lot to the other non-blockchain and LLMs and frontends in this organization so at the end I would like to say Aossie has been providing me a bigger lay-ground and open environment where freedom to contribute , and learn meet peoples and get your skills nurished by mentors. so I really have enjoyed this contribution period and I would like to continue this journey and made this as a part of my engineering acheivements.		

A NOTE OF GRATITUDE

First of all **thank you — every mentor, maintainer, and reviewer** who took the time to look at my pull requests, leave a comment, suggest an improvement, or simply merge a fixes.

When I started contributing to AOSSIE, StabilityNexus, and DjedAlliance, I was a student who knew Solidity from books and lectures but had never experienced anything real on a live blockchain apart from some basic blockchain projects. Every code review even the tough ones was a lesson. Every merged PR was a small proof that I could actually build things that matter.

Over these four months I opened **114 pull requests** and **got 77 merged**, solved real issues in production-grade DeFi contracts, Web3 frontends, and ML tooling and learned far more from the community than I never could have alone.

What I found here people who genuinely care about **open source, clean code, and guiding**. This culture helped me to how to think about software. I plan to carry it forward into GSoC, into every future PR, and into whatever I build next.

I want to spend this summer turning that understanding into a protocol that outlives GSoC, something the StabilityNexus community actually uses, extends, and builds on.

In this organization I have spent around 4 months dedicatedly and learned a lot, gaining hands-on experience in real-world development, improving my problem-solving skills, and contributing meaningfully to impactful projects, and I would love to continue this journey with you all.

I am not done yet. This is the beginning.

Aniket Kumar

B.E. Information Technology · Panjab University, Chandigarh · 3rd Year
iamaniketkumarmaner@gmail.com · github.com/aniket866 · Discord: aniket866

Thank you for the opportunity. I will make it count.